

# 一种面向时空可组合性的编程范式

Yifan Shi<sup>1,2</sup> Wei Zhang<sup>1</sup> Tianyi Cui<sup>2</sup>

<sup>1</sup> 北京大学 <sup>2</sup>DeepSeek-AI

英文原题: *A Programming Paradigm for Spatiotemporal Composability*

来源文件: `doc/deepseek-group/cordis-paper.pdf`

日期说明: 源 PDF 元数据显示, 其生成与修改日期均为 2026 年 7 月 31 日; 该日期仅表示源 PDF 文件的生成与修改时间, 源文件未提供可确认的论文发表日期或版本号。

## 摘要

现代软件——从插件系统到自演化智能体框架——日益需要动态组合, 但其形式化基础仍不完善。本文识别出这一问题的两个正交维度: 时间可组合性, 即在移除组件时完全逆转该组件副作用的能力; 空间可组合性, 即声明并以反应式方式管理组件间依赖关系的能力。本文分别将经典的效应与余效应概念提升为运行时机制, 以应对这两个维度。本文将可回退效应形式化: 每个上下文变换均配有显式逆变换; 并证明效应跟踪保持组合运算, 从而保证组件移除时状态得以完全恢复。本文将反应式余效应形式化: 依赖项注册在类型化上下文中, 依赖满足性的变化会触发组件的自动激活与停用。组件生命周期模型将可回退效应和反应式余效应与多种控制流之间的交互操作化, 由此得到一种统一的上下文类型, 将效应上下文与余效应上下文整合为连贯的编程范式。本文在 Cordis 中实现了这些理念。Cordis 是一个面向时空可组合性的元框架, 提供具有效应跟踪和余效应解析功能的核心库, 以及具有配置协调与热模块替换功能的声明式组件加载器。

# 目录

|          |                     |           |
|----------|---------------------|-----------|
| <b>1</b> | <b>引言</b>           | <b>4</b>  |
| 1.1      | 可组合性的维度             | 4         |
| 1.2      | 动机示例                | 4         |
| 1.2.1    | 插件系统                | 4         |
| 1.2.2    | 自演化智能体框架            | 5         |
| 1.2.3    | 粗粒度变通方案             | 5         |
| 1.3      | 贡献                  | 5         |
| <b>2</b> | <b>预备知识</b>         | <b>6</b>  |
| 2.1      | 效应                  | 6         |
| 2.2      | 余效应                 | 6         |
| 2.3      | 与动态可组合性的关系          | 7         |
| <b>3</b> | <b>可回退效应与反应式余效应</b> | <b>7</b>  |
| 3.1      | 可回退效应               | 7         |
| 3.1.1    | 效应上下文               | 8         |
| 3.1.2    | 可回退效应函数             | 9         |
| 3.2      | 反应式余效应              | 11        |
| 3.2.1    | 余效应上下文              | 11        |
| 3.2.2    | 规格与通知               | 12        |
| 3.2.3    | 隔离与拦截               | 12        |
| 3.3      | 组件生命周期              | 14        |
| 3.3.1    | 幂等恢复                | 15        |
| 3.3.2    | 迭代                  | 15        |
| 3.3.3    | 连贯性                 | 16        |
| 3.3.4    | 异步性                 | 17        |
| 3.4      | 上下文范式               | 18        |
| 3.4.1    | 统一上下文               | 18        |
| 3.4.2    | 上下文范式的定位            | 19        |
| <b>4</b> | <b>实现与案例研究</b>      | <b>19</b> |
| 4.1      | 核心库                 | 20        |
| 4.1.1    | 效应跟踪                | 20        |
| 4.1.2    | 余效应操作               | 21        |
| 4.1.3    | 组件生命周期              | 22        |
| 4.1.4    | 上下文访问               | 23        |
| 4.2      | 组件加载器               | 25        |
| 4.2.1    | 声明式配置层              | 25        |
| 4.2.2    | 热模块替换               | 26        |
| 4.3      | 案例研究: Koishi        | 27        |
| <b>5</b> | <b>讨论</b>           | <b>28</b> |
| 5.1      | 服务多路复用              | 28        |
| 5.2      | 访问控制与沙箱             | 29        |

|          |             |           |
|----------|-------------|-----------|
| 5.3      | 语言独立性与选择    | 30        |
| 5.4      | 相互依赖与组件粒度   | 31        |
| 5.5      | 依赖类型与版本管理   | 31        |
| <b>6</b> | <b>相关工作</b> | <b>32</b> |
| 6.1      | 效应与余效应系统    | 32        |
| 6.2      | 编程范式        | 33        |
| 6.3      | 时间可组合性      | 34        |
| 6.4      | 空间可组合性      | 35        |
| <b>7</b> | <b>结论</b>   | <b>36</b> |
| <b>A</b> | <b>术语表</b>  | <b>44</b> |
| A.1      | 可组合性的维度     | 44        |
| A.2      | 可回退效应       | 44        |
| A.3      | 反应式余效应      | 45        |
| A.4      | 生命周期与上下文范式  | 45        |
| A.5      | Cordis 实现术语 | 46        |

## 1 引言

组合，也就是由较简单的部分组装出复杂系统，是软件工程的一项基础原则 [1]。传统上，组合是静态的：函数调用、模块导入和类继承在编译时解析，并在整个执行过程中保持固定。然而，现代软件日益需要动态组合，即在运行时加载、卸载和重新配置组件。插件架构 [2] 和自演化智能体框架都要求系统能够即时、安全地添加和移除功能，但当前实践仍依赖粗粒度机制 [3]，只能通过重启进行重新配置，并会丢弃运行时状态。尽管动态组合在实践中日益重要，其理论基础仍不充分，远不及静态组合所拥有的丰富形式化框架。

### 1.1 可组合性的维度

为了刻画动态组合的要求，除了已有深入研究的组合之代数性质外，本文还辨识出两个相互正交的维度：

- **时间可组合性**关注时间维度：移除组件时，必须完整且安全地回退该组件对共享环境所做的修改。这要求跟踪组件执行的每一次资源分配、事件注册和状态变更，并保证在移除组件时有序回收这些内容。
- **空间可组合性**关注空间维度：组件必须能够以结构化且可验证的方式声明、发现并解析彼此之间的依赖。这要求管理依赖拓扑，并在依赖发生变化时协调组件生命周期。

在静态环境中，时间可组合性可归结为词法作用域，例如 RAII[4] 和括号模式 [5]；空间可组合性则可归结为模块导入解析。在动态环境中，组件会在运行时加入和退出，这两个维度都变得困难得多：时间可组合性必须处理长期存续且带有状态的效应，而这些效应的作用域并不受词法边界约束；空间可组合性必须处理在执行期间出现、消失或改变身份的依赖。

### 1.2 动机示例

#### 1.2.1 插件系统

插件系统是动态组合的典型实例。本文以应用最广泛的可扩展集成开发环境之一 Visual Studio Code (VSCode) 作为代表性示例。

**时间限制。**VSCode 在一个名为扩展宿主的共享进程中运行所有扩展。尽管扩展可以动态安装，该宿主却没有在运行时卸载单个扩展代码的机制。一旦某个扩展的 `activate` 函数执行完毕，停用或卸载该扩展就需要重启整个宿主，并会影响所有已加载的扩展。主题、快捷键绑定和代码片段等纯声明式扩展不含代码，因此可以自由移除；然而，按安装量排名前 100 的扩展中，有 87 个含有可执行代码<sup>1</sup>，移除时因而需要重启。VSCode 虽然提供了 `deactivate` 钩子，但它只是在宿主进程已经开始终止时调用的优雅关闭回调，因此并不能实现实时移除。此外，该钩子将效应的释放与效应的创建（位于 `activate` 中）分离，违背了关注点局部性，也使完整清理难以验证。

**空间限制。**VSCode 确实提供了 `extensionDependencies`，用于声明扩展之间的依赖，但这一机制很少使用：按安装量排名前 100 的扩展中，只有 7 个会通过 `extensionDependencies` 声明对非内置扩展的依赖<sup>1</sup>。这种稀少性反映了扩展 API 的形态：该 API 只暴露命令、视图和语言功能等固定的表层扩展点。扩展通过这些扩展点向宿主贡献功能，而不是相互依赖，因此扩展间依赖很少出现。此外，VSCode 的扩展间交互机制不提供结构化契约：它通过 `vscode.extensions.getExtension(...).exports` 向其他扩展暴露某个扩展的功能，但返回值没有类型（默认为 `any`），因此依赖方无法依靠经过类型检查的接口。简言之，VSCode 引导扩展使用一组固定的、由宿主提供的扩展点，却没有为扩展彼此依赖提供安全且结构化的方式。

<sup>1</sup>数据于 2026 年 6 月 9 日从 Visual Studio Code Marketplace 获取。

这两项限制并非 VSCode 独有；它们在各种插件系统中都会出现，只是程度有所不同 [2, 6]。

### 1.2.2 自演化智能体框架

现代 AI 智能体依赖运行时智能体框架 [7, 8, 9]。这些系统可以组合各类工具套件 [10] 和执行环境，管理权限与沙箱，维持会话状态与持久化，提供上下文管理与记忆系统 [11]，编排子智能体和多智能体工作流 [12]，并向用户与自动化系统提供接口。未来的智能体框架可能会在持续处理请求的同时，生成并部署对自身组件的修改。由模型合成的可复用工具，是组件级自修改的一种范围更窄的先行形态 [13]。每一次这样的修改，本身都是一个动态组合实例。

由于这些修改持续发生，而且只有有限乃至完全没有人工监督，动态可组合性便不可或缺。如果缺少时间可组合性，每次自修改都会迫使整个进程重启，并丢弃在进程本地积累的全部状态；以这样的频率发生时，累积的不可用时间会相当可观，执行中的任务也会一再中断；更糟的是，有缺陷的自修改可能使恢复所必需的进程本身失效。如果缺少空间可组合性，每个模块都必须自行检测它所依赖模块的变化，并在这些模块出现、消失或改变身份时作出适应，而它只能通过临时拼凑的手段完成这些工作；更糟的是，朴素的代码替换策略可能悄无声息地破坏依赖方，或者引入只会在重新加载时才暴露的循环依赖。

### 1.2.3 粗粒度变通方案

动态可组合性在形式化研究中得到的关注有限，原因之一是操作系统和容器编排器已经提供了一种粗粒度替代方案。终止并重启进程，可以在整个地址空间的粒度上实现时间可组合性；容器编排 [3] 则可以在整个服务的粒度上实现空间可组合性。在实践中，大多数软件依靠这些粗粒度机制来容忍细粒度可组合性的缺失：模块行为异常时重启进程，服务依赖则交由编排器管理。

然而，这一变通方案代价高昂。在时间维度上，每次重启都会丢弃进程本地积累的全部状态，例如缓存、连接和部分完成的计算；重建这些状态需要数秒乃至数分钟 [14]。在此期间维持可用性需要冗余副本，以资源开销来弥补无法恢复单个组件的问题。在空间维度上，容器级编排无法表达共享同一地址空间的组件之间的依赖，还会让原本可用本地函数调用完成的交互承担网络开销。这两种机制都在进程和容器的边界上运作，但现代系统越来越多地在更细的层次上进行组合。这种粒度不匹配要求一种组合式抽象，使其能在与组件本身相同的层次上管理效应和依赖。

## 1.3 贡献

动态可组合性的两个维度分别涉及计算如何修改环境，以及计算如何依赖环境。这两个方向恰好是效应系统 [15, 16] 与余效应系统 [17, 18] 所形式化的内容：效应为推理环境修改提供形式化词汇，而余效应为推理环境需求提供形式化词汇。然而，现有表述仅限于在词法上固定的作用域内进行编译时分析，无法扩展至组件在运行时加入和退出的动态场景。通过将效应提升为可回退的运行时模型，并将余效应提升为反应式依赖解析机制，本文为动态可组合性建立了统一的形式化基础。该基础与语言无关，适用于任何需要动态组合的软件架构。本文作出如下贡献：

1. 本文形式化了可回退效应（第 3.1 节）：在该模型中，每个上下文变换都与一个显式逆函数配对，由此得到的效应函数具有保持组合结构的跟踪与恢复操作。这为动态时间可组合性奠定了代数基础。
2. 本文形式化了反应式余效应（第 3.2 节）：在一个有类型的依赖上下文中，组件以依赖集的形式声明其需求；基于满足性的通知机制会自动将状态转变划分为激活型、停用型或中性。这为动态空间可组合性奠定了代数基础。
3. 本文建立了一个组件生命周期模型（第 3.3 节），使可回退效应与反应式余效应同各种控

制流之间的交互得以操作化；本文还将效应上下文与余效应上下文整合为统一的上下文类型（第 3.4 节），由此构成一种面向动态可组合系统的编程范式。

4. 本文在 Cordis（第 4 节）中实现了这些理念。Cordis 是一个时空可组合性元框架，其核心库以效应跟踪和余效应解析实现了形式化模型；它还提供声明式组件加载器，支持配置协调与热模块替换。本文通过 Koishi 聊天机器人平台上的案例研究验证了这一设计；Koishi 生态拥有超过 4000 个在生产环境中使用的社区插件。

## 2 预备知识

本节简要概述效应系统与余效应系统——二者是支撑本文工作的两大理论支柱。本文假定读者熟悉基本类型论与范畴论；此处的目标是约定记号，并介绍第 3 节将操作化为运行时机制的核心抽象。

### 2.1 效应

在简单类型 lambda 演算 (simply typed lambda calculus, STLC) [19, 20] 中, 类型判断  $\Gamma \vdash t : T$  表明在上下文  $\Gamma$  下项  $t$  具有类型  $T$ 。效应系统对类型加以细化，以记录计算可能产生哪些副作用，从而得到如下形式的判断：

$$\Gamma \vdash t : T_{\text{effect}} \quad (1)$$

这里，结果类型由效应代数中的一个元素标注，该元素记录计算可能产生哪些副作用，从而支持对有状态计算进行组合式推理。这一方法源于 Lucassen 和 Gifford[21]；两人引入了一种带种类的类型系统，通过区分类型、效应和区域来发现并行程序中的调度约束。

**单子效应。** Moggi[15] 率先通过单子从范畴论角度对计算效应进行建模；Wadler[22] 则在 Haskell 中推广了这一方法。范畴  $\mathcal{C}$  上的单子  $(T, \eta, \mu)$  将含效应计算封装为类型  $T(A)$  的值，其中  $\eta : A \rightarrow T(A)$  将纯值提升至该类型，而  $\mu : T(T(A)) \rightarrow T(A)$  对嵌套计算进行定序。经典实例包括 Maybe 单子（部分性）、State 单子（可变状态）和 IO 单子（外部交互）。

**代数效应。** Plotkin 和 Power[16, 23] 表明，代数操作决定单子，由此建立了一个将效应接口与其实现解耦的框架。效应签名  $\Sigma$  声明一组操作（例如，对于状态， $\text{get} : () \rightarrow S$ ,  $\text{put} : S \rightarrow ()$ ）；程序可以自由调用这些操作，而无须预先确定某种具体解释。Plotkin 和 Pretnar[24] 随后引入了效应处理器，通过提供延续语义来解释操作：

$$\text{handle } e \text{ with } \{ \text{op}(v, \kappa) \mapsto \dots \} \quad (2)$$

处理器接收操作参数  $v$  和定界延续  $\kappa$ ；处理器可以调用该延续零次、一次或多次，从而在统一框架内实现异常、协程和非确定性 [25]。Koka[26, 27]、Eff[28] 和 OCaml 5[29] 等语言均采用了代数效应，各自作出了不同的设计权衡。

### 2.2 余效应

与效应对偶，余效应系统 [17, 30] 丰富的是上下文而非类型，从而得到如下形式的判断：

$$\Gamma_{\text{coeffect}} \vdash t : T \quad (3)$$

这里，上下文由余效应代数中的一个元素标注，该元素描述计算对其环境的要求，例如需要访问的资源、需要具备的能力或需要依赖的服务。效应刻画程序对世界的影响，而余效应则



刻画世界对程序的约束。

**余单子余效应。** 最早由 Uustalu 和 Vene[31] 提出使用余单子来构造上下文依赖计算；两人提出以对称（半）么半余单子作为 Moggi 单子效应框架的对偶，从而刻画数据流和属性求值等概念。Petricek 等 [17] 在此基础上提出将余效应作为上下文依赖性的统一静态分析方法。余单子  $(D, \varepsilon, \delta)$  刻画上下文依赖计算： $\varepsilon : D(A) \rightarrow A$  从上下文中抽取当前值， $\delta : D(A) \rightarrow D(D(A))$  则复制上下文，以供嵌套访问。环境余单子  $D(X) = E \times X$  刻画对固定环境  $E$  的依赖；流余单子  $D(X) = \mathbb{N} \rightarrow X$  则刻画对时序数据的依赖。

**分级余效应。** 为了实现更细粒度的跟踪，分级余效应系统使用预序半环  $\mathcal{S} = (S, \leq, +, \times, 0, 1)$  作为余效应代数 [32]；Gabori 等 [18] 后来将这一体系与分级效应统一起来。 $S$  中的元素标注每一个变量绑定，以量化其使用情况：0 表示未使用，1 表示线性使用， $n$  表示有界使用， $\infty$  表示不受限使用。半环运算分别以顺序方式（ $\times$ ）和并行方式（ $+$ ）组合余效应，从而在一个统一的代数框架 [36] 内实现精确的资源跟踪、敏感度分析 [33] 和信息流控制 [34, 35]。

### 2.3 与动态可组合性的关系

效应系统与余效应系统沿两个互补方向组织对计算的推理：效应描述计算如何修改其环境，余效应则描述计算如何依赖其环境。这两个方向对应第 1 节所确定的动态可组合性的两个维度：

- **时间可组合性**要求组件对共享环境的修改在组件卸载时可逆。这里相关的是有状态效应，因为它们会持久地改变该环境；要逆转这样的变换，该变换就必须存在逆变换。
- **空间可组合性**要求声明组件间的依赖，并以反应式方式管理这些依赖。这些依赖正是余效应所捕获的内容；管理这些依赖，就是根据环境所提供的内容逐一解析它们。

然而，经典的效应系统与余效应系统是静态工具：效应在词法上固定的作用域内得到跟踪，并由编译期处理器消去；余效应标注则依据执行前已经确定的上下文进行验证。相比之下，动态组合要求这些保证适用于在运行时加入和离开的组件，并且面对持续演化的上下文。固定的词法作用域无法界定部署后才加载的插件；编译期上下文也无法预见由运行时配置产生的依赖。

这促使本文转换视角：不再通过增加更多标注来扩展静态类型系统，而是将效应和余效应的概念结构具体化，使运行时能够直接对其进行操作，从而动态建立这些系统在静态情况下提供的保证。第 3 节将展开这一构造。

## 3 可回退效应与反应式余效应

本节把第 2 节介绍的效应与余效应概念提升为运行时机制，从而构建一种动态组合理论。其核心思想是把承载效应与余效应的类型上下文转化为上下文类型，即一种可在运行时操作、将上下文本身具象化为一等实体的类型。对于效应类型，将其建模为配备逆变换的上下文变换，从而实现动态的时间可组合性。对于余效应上下文，则将其建模为携带依赖信息的类型，从而实现动态的空间可组合性。这两种具有不同控制流的机制相互作用，产生了组件生命周期模型；而同时承载效应与余效应的统一上下文本身便构成了一种编程范式。

### 3.1 可回退效应

**时间可组合性**是指在运行时加载和卸载组件，并在卸载时使共享环境恢复到组合前状态的能力。这要求组件对环境所作的每项修改都既可跟踪又可逆。因此，将效应建模为类型为  $\Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)$  的函数：把它应用于当前上下文后，会得到修改后的上下文以及一个显式逆

函数。这个逆函数使效应可逆，而把它返回给运行时则使效应可跟踪。本文称这类效应为可回退效应：执行期间记录并组合这些逆函数后，完整恢复环境便成为一种结构性保证。

### 3.1.1 效应上下文

给定任意非纯函数  $f_{\text{impure}}: X \rightarrow Y$ ，将其变换为纯函数形式  $f: \Gamma \times X \rightarrow \Gamma \times Y$ ，其中  $\Gamma$  是上下文，所有可能的副作用均可表示为  $\Gamma$  上的变换。对于任意固定输入  $x: X$ ，诱导映射  $\gamma \mapsto \text{pr}_1(f(\gamma, x))$  捕获了  $f$  的副作用，而与返回值无关。作为初步近似，假设每个效应的逆都能预先获得；于是， $\Gamma$  上可接受的效应在复合运算。下构成子群  $\mathfrak{F}_\Gamma \leq \text{Sym}(\Gamma)$ ，其公理对应于系统的以下结构要求：

- 封闭性保证效应的顺序复合仍然可接受；
- 单位元  $\text{id}_\Gamma$ ，即  $\Gamma$  上的恒等函数，是复合运算的单位元；
- 逆元要求每个效应  $f \in \mathfrak{F}_\Gamma$  都有一个回退函数  $f^{-1} \in \mathfrak{F}_\Gamma$ ，从而保证任意效应复合均可撤销。

对逆元的要求源于一项作用域限制： $\Gamma$  只对系统的内部状态建模，也就是系统拥有完全控制权的状态。系统对自身上下文所作的每项修改都是可逆的，因为先前的值总能恢复。以系统边界之外的实体为目标的操作（网络请求、文件 I/O）并不修改  $\Gamma$ ；从  $\mathfrak{F}_\Gamma$  的角度看，它们的作用等同于  $\text{id}_\Gamma$ 。外部交互由领域特定的策略处理（例如补偿事务 [37]），不在本模型的讨论范围内。

为了在上下文本身中记录效应，给出如下核心定义。

**定义 1.** 给定上下文  $\Gamma$ ，其效应上下文定义为

$$\partial\Gamma := \Gamma \times \mathfrak{F}_\Gamma. \quad (4)$$

它可以理解为二元组  $(\gamma, \varphi)$ ，其中：

- $\gamma \in \Gamma$  表示当前的上下文状态；
- $\varphi \in \mathfrak{F}_\Gamma$  是将上下文恢复到初始状态的变换。

特别地，初始状态可表示为  $(\gamma_0, \text{id}_\Gamma)$ 。

由于有上述  $\varphi$ ，在  $\partial\Gamma$  上执行的所有效应都能被自动跟踪并完整恢复。下面给出具体构造。

**定义 2.** 定义从  $\mathfrak{F}_\Gamma$  到  $\mathfrak{F}_{\partial\Gamma}$  的变换  $\text{track}_\Gamma$ ：

$$\begin{aligned} \text{track}_\Gamma: (\Gamma \rightarrow \Gamma) &\rightarrow \partial\Gamma \rightarrow \partial\Gamma, \\ \text{track}_\Gamma = f &\mapsto (\gamma, \varphi) \mapsto (f(\gamma), \varphi \circ f^{-1}). \end{aligned}$$

这个变换把上下文  $\Gamma$  上的任意函数  $f \in \mathfrak{F}_\Gamma$  转化为效应上下文  $\partial\Gamma$  上的函数。在状态  $(\gamma, \varphi)$  中执行  $\text{track}_\Gamma(f)$  时， $f$  的逆与  $\varphi$  复合，从而在上下文中有用地跟踪  $f$  的效应。每个  $\text{track}_\Gamma(f)$  本身都是  $\partial\Gamma$  上的双射，其逆为  $(\gamma', \varphi') \mapsto (f^{-1}(\gamma'), \varphi' \circ f)$ ；因此，将  $\partial\Gamma$  上的可接受效应定义为像  $\mathfrak{F}_{\partial\Gamma} := \text{track}_\Gamma(\mathfrak{F}_\Gamma)$ 。由定理 4，它是  $\text{Sym}(\partial\Gamma)$  的一个子群，并且与  $\mathfrak{F}_\Gamma$  同构。

**定理 3.** 下图交换。也就是说，对于任意  $f \in \mathfrak{F}_\Gamma$ ，都有  $\text{pr}_1 \circ \text{track}_\Gamma(f) = f \circ \text{pr}_1$ 。

$$\begin{array}{ccc} \Gamma & \xrightarrow{f} & \Gamma \\ \text{pr}_1 \uparrow & & \uparrow \text{pr}_1 \\ \partial\Gamma & \xrightarrow{f'} & \partial\Gamma \end{array} \quad f' = \text{track}_\Gamma(f).$$



**定理 4.**  $\text{track}_\Gamma$  保持  $\circ$  运算。也就是说，对于任意  $f, g \in \mathfrak{F}_\Gamma$ ,

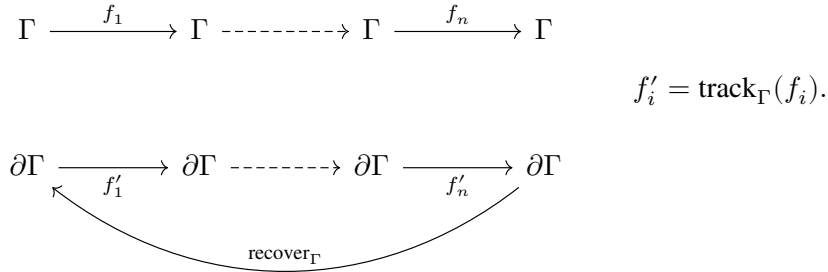
$$\text{track}_\Gamma(f \circ g) = \text{track}_\Gamma(f) \circ \text{track}_\Gamma(g). \quad (5)$$

这一定理保证，对于  $\mathfrak{F}_\Gamma$  中的每个函数，都能构造一个等价的  $\text{track}$  版本，使其效应在  $\partial\Gamma$  上得到跟踪。

**定义 5.** 在  $\partial\Gamma$  上定义变换  $\text{recover}_\Gamma$ :

$$\begin{aligned} \text{recover}_\Gamma &: \partial\Gamma \rightarrow \partial\Gamma, \\ \text{recover}_\Gamma &= (\gamma, \varphi) \mapsto (\varphi(\gamma), \text{id}_\Gamma). \end{aligned}$$

这个变换把恢复函数  $\varphi$  应用于当前状态  $\gamma$ ，并将  $\varphi$  重置为恒等函数，从而恢复施加于  $\gamma$  的全部效应。下图说明：在  $\partial\Gamma$  上依次应用效应  $\text{track}_\Gamma(f_1), \dots, \text{track}_\Gamma(f_n)$  后， $\text{recover}_\Gamma$  如何使上下文回到初始状态：



### 3.1.2 可回退效应函数

上一小节的  $\text{track} / \text{recover}$  模型假设逆函数已经给出，并以原子方式恢复所有累积的效应。然而在实践中，每个效应的逆函数并非预先已知：它必须由调用者在应用效应时提供。此外， $\text{recover}$  是全有或全无的；它无法在保留其他效应的同时选择性地撤销某个效应。为解决这两个问题，在输入端与输出端同时增强该模型：

1. 在输入端，不仅变换  $\Gamma$ ，还返回相应的逆函数，从而获得手动跟踪能力： $\Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)$ ，即  $\Gamma \rightarrow \partial\Gamma$ ；
2. 在输出端，不仅变换  $\partial\Gamma$ ，还返回相应的逆函数，从而获得部分恢复能力： $\partial\Gamma \rightarrow \partial\Gamma \times (\partial\Gamma \rightarrow \partial\Gamma)$ ，即  $\partial\Gamma \rightarrow \partial^2\Gamma$ 。

这一增强保持了输入与输出之间的结构一致性，因而仍可建立保持  $\text{track}$  数学性质的相应理论。将  $\mathfrak{F}_\Gamma$  增强为  $\mathfrak{E}_\Gamma$  与  $\mathfrak{E}_\Gamma^*$ 。

**定义 6.** 效应函数  $\mathfrak{E}_\Gamma$  与严格效应函数  $\mathfrak{E}_\Gamma^*$  定义为

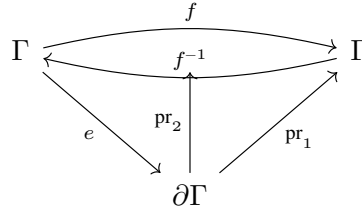
$$\begin{aligned} \mathfrak{E}_\Gamma &:= \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma), \\ \mathfrak{E}_\Gamma^* &:= (e: \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)) \\ &\quad \times \left( (\gamma: \Gamma) \rightarrow (\text{let } (\delta, g) = e(\gamma) \text{ in } g(\delta) = \gamma) \right). \end{aligned} \quad (6)$$

其中， $e(\gamma)$  给出二元组  $(\delta, g)$ ，表示：

- $\delta: \Gamma$  是新的上下文；
- $g: \Gamma \rightarrow \Gamma$  是当前效应的逆函数。

上式中的约束  $g(\delta) = \gamma$  可由以下交换图表示；该图保证返回值的第二个分量确实是变换

本身的逆:



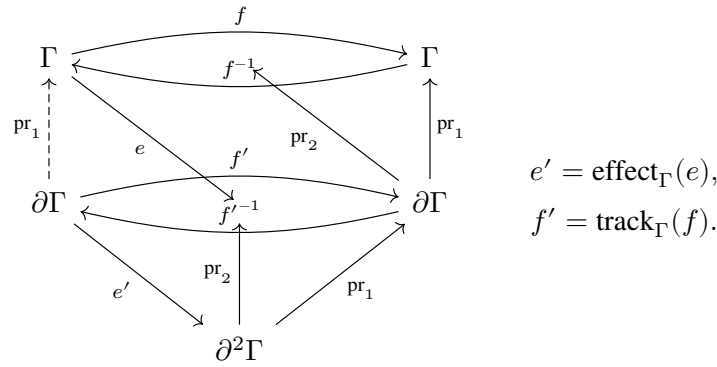
正如 track 把  $\mathfrak{F}_\Gamma$  提升到  $\mathfrak{F}_{\partial\Gamma}$ , 定义 effect 将  $\mathfrak{E}_\Gamma$  提升到  $\mathfrak{E}_{\partial\Gamma}$ 。

定义 7. 效应函数变换  $\text{effect}_\Gamma$  定义为

$$\text{effect}_\Gamma: \mathfrak{E}_\Gamma \rightarrow \partial\Gamma \rightarrow \partial^2\Gamma,$$

$$\text{effect}_\Gamma = e \mapsto (\gamma, \varphi) \mapsto \left( \text{let } (\delta, g) = e(\gamma) \text{ in } \left( (\delta, \varphi \circ g), \text{track}_\Gamma(g) \right) \right).$$

下图说明 effect 如何把  $\mathfrak{E}_\Gamma$  上的操作“翻译”为  $\mathfrak{E}_{\partial\Gamma}$  上的操作。下文将证明, 这一翻译确实保持图的交换性。



定理 8. effect 在  $\mathfrak{E}^*$  上封闭。也就是说, 对于任意  $e \in \mathfrak{E}_\Gamma^*$ , 都有  $\text{effect}_\Gamma(e) \in \mathfrak{E}_{\partial\Gamma}^*$ 。

效应函数  $\mathfrak{E}_\Gamma$  不再是上下文上的自同态, 因而不能直接复合。因此, 定义一种新的运算来表示效应复合。

定义 9. 给定函数  $f, g \in \mathfrak{E}_\Gamma$ , 其效应复合  $f \diamond g$  定义为

$$f \diamond g: \Gamma \rightarrow \partial\Gamma,$$

$$f \diamond g = \gamma \mapsto \left( \text{let } (\delta, s) = g(\gamma) \text{ in } \left( \text{let } (\varepsilon, t) = f(\delta) \text{ in } (\varepsilon, s \circ t) \right) \right).$$

定理 10.  $\diamond$  运算在  $\mathfrak{E}_\Gamma^*$  上封闭。也就是说, 对于任意  $f, g \in \mathfrak{E}_\Gamma^*$ , 都有  $f \diamond g \in \mathfrak{E}_\Gamma^*$ 。

现在可以证明 effect 具有与 track 类似的性质。

定理 11. effect 保持  $\diamond$  运算。也就是说, 对于任意  $f, g \in \mathfrak{E}_\Gamma$ ,

$$\text{effect}_\Gamma(f) \diamond \text{effect}_\Gamma(g) = \text{effect}_\Gamma(f \diamond g). \quad (7)$$

上述构造共同组成了可回退效应： $\mathfrak{E}_T^*$  中的每个效应函数都显式提供自身的逆；effect 在效应上下文  $\partial\Gamma$  上跟踪这些逆； $\diamond$  运算则在保持可回退性的同时复合这些效应。该构造实现了时间可组合性：加载组件相当于应用一系列效应函数（把逆累积到  $\varphi$  中），卸载组件则相当于应用  $\varphi$ ，使上下文恢复到组合前的状态。

卸载一个组件是否会干扰另一个组件，取决于  $\diamond$  下的交换性：若  $f \diamond g = g \diamond f$ ，则恢复  $f$  时会保留  $g$  的贡献，这两个效应便可以安全地存在于彼此独立的组件中。当效应不交换时，系统必须施加顺序约束：或者在单个组件内部施加约束（通过效应迭代器的后进先出纪律，见第 3.3.2 节），或者在组件之间施加约束（通过反应式余效应规定由依赖驱动的激活顺序，见第 3.2 节）。

### 3.2 反应式余效应

空间可组合性是指组件能够相互声明依赖，并由系统在运行时解析、提供和撤销这些依赖。为此，每当共享上下文发生变化时，都必须重新评估依赖满足性，使组件恰好在其依赖变得可用时激活，并在依赖被撤销时停用。因此，将组件的依赖建模为一个规格，并依据该规格，把上下文的每次变化分类为激活型、停用型或中性。依据规格进行分类，可以检测满足性是否发生变化；对分类结果作出响应，则会驱动激活与恢复。这样的余效应称为反应式余效应：通过对上下文变化进行分类，并由此驱动激活与恢复，正确的依赖顺序便成为一种结构性保证。

#### 3.2.1 余效应上下文

传统的控制反转 (IoC) 容器 [38, 39] 通常将依赖建模为简单的键值映射。本节将 IoC 形式化为一种余效应上下文；它与可回退效应协同作用，为动态组合提供数学基础。

**定义 12.** 给定类型族  $\mathcal{V} : K \rightarrow \text{Type}$ ，将余效应上下文定义为如下依赖偏函数类型：

$$\Sigma := (k : K) \rightarrow \mathcal{V}_k. \quad (8)$$

其中， $\sigma : \Sigma$  是有限偏函数，它为每个  $k \in \text{dom}(\sigma) \subseteq K$  指派一个类型为  $\mathcal{V}_k$  的值。采用如下记法：

- $\sigma(k)$  表示函数应用（当  $k \in \text{dom}(\sigma)$  时有定义）；
- $\sigma[k \mapsto v]$  表示扩张（当  $k \notin \text{dom}(\sigma)$  时有定义）；
- $\sigma \setminus k$  表示限制（当  $k \in \text{dom}(\sigma)$  时有定义）；
- $k \in \text{dom}(\sigma)$  表示成员关系。

使用类型族  $\mathcal{V}$  可以确保每个依赖键  $k$  都关联到特定的值类型  $\mathcal{V}_k$ ，从而为依赖访问提供静态类型安全。扩张与限制的前置条件对应于可回退效应中的幂等性要求：同一个依赖不能被提供两次（扩张要求  $k \notin \text{dom}(\sigma)$ ），不存在的依赖也不能被撤销（限制要求  $k \in \text{dom}(\sigma)$ ）。基于这一上下文结构，定义两个核心操作。

**定义 13.**  $\Sigma$  上的 get 与 set 操作定义如下：

$$\begin{aligned} \text{get} &: (k : K) \rightarrow \Sigma \rightarrow \mathcal{V}_k, \\ \text{get} &= k \mapsto \sigma \mapsto \sigma(k), \\ \text{set} &: (k : K) \times \mathcal{V}_k \rightarrow \Sigma \rightarrow \Sigma \times (\Sigma \rightarrow \Sigma), \\ \text{set} &= (k, v) \mapsto \sigma \mapsto (\sigma[k \mapsto v], \lambda \sigma'. \sigma' \setminus k). \end{aligned}$$

其中， $\text{get}(k)$  的前置条件为  $k \in \text{dom}(\sigma)$ ， $\text{set}(k, v)$  的前置条件为  $k \notin \text{dom}(\sigma)$ 。

值得注意的是,  $\text{set}(k, v)$  的类型为  $\mathfrak{E}_\Sigma$ , 恰好是余效应上下文上的效应函数。因此可以直接应用第 3.1 节的效应机制:  $\text{effect}_\Sigma$  会自动跟踪并恢复依赖注册。这正是反应式余效应与可回退效应之间的协同关系: 余效应操作是效应, 而效应是可回退的。

### 3.2.2 规格与通知

上述定义说明了如何注册和访问单个依赖。然而, 访问一个不存在的依赖会导致运行时失败。因此, 组件应当等到其声明的全部依赖都存在时才激活, 而不应乐观地访问依赖, 并在缺少某项依赖时失败。这引出了两个问题: 组件声明的依赖是否共同得到满足, 以及当满足状态变化时系统应如何响应。余效应上下文  $\Sigma$  具有一种自然的观测结构, 使这两个问题都易于处理: 对于任意余效应规格  $d \subseteq K$ , 定义满足谓词

$$\sigma \models d := \forall k \in d. k \in \text{dom}(\sigma). \quad (9)$$

该谓词是可判定的 (因为  $\text{dom}(\sigma)$  是有限集)。由于对  $\sigma$  的所有变更都经由效应函数完成 (而这些函数的逆函数会恢复此前的定义域), 所以每个效应边界上的满足性变化均可被检测。这就是反应性的代数基础: 效应系统保证每一次余效应变化都会被观察到。

**定义 14.** 余效应规格定义为

$$\mathfrak{D}_\Sigma := \text{Set}(K), \quad (10)$$

表示组件向环境声明的依赖项集合。

使该规格具有反应性的, 是它对状态迁移进行分类的方式。任一把  $\sigma$  变换为  $\sigma'$  的效应, 都可以由规格  $d \in \mathfrak{D}_\Sigma$  根据  $d$  的满足状态是否改变来分类。

**定义 15.** 给定余效应规格  $d \subseteq K$  以及状态  $\sigma, \sigma' \in \Sigma$ , 定义

$$\text{notify}_d(\sigma, \sigma') := \begin{cases} \text{激活型,} & \text{若 } \sigma \not\models d \wedge \sigma' \models d, \\ \text{停用型,} & \text{若 } \sigma \models d \wedge \sigma' \not\models d, \\ \text{中性,} & \text{其他情况.} \end{cases} \quad (11)$$

由于  $\sigma \models d$  是可判定的, 且所有状态迁移均由效应函数介导, 所以上述定义是良定义的。反应式不变量为: 激活型迁移触发组件效应的执行 (并进行完整的效应跟踪), 停用型迁移则触发已累积效应的恢复。这些迁移与控制流相互作用时, 其精确的操作语义取决于具体情况, 并将在第 3.3 节中展开。

空间可组合性来自  $\text{set}$  与  $\text{notify}$  之间的相互作用。若组件  $A$  通过  $\text{set}(k, v)$  提供键  $k$ , 而组件  $B$  声明  $k \in d_B$ , 那么  $B$  只有在  $A$  完全激活并提供  $k$  之后才能激活 (因为  $\sigma \models d_B$  要求  $k \in \text{dom}(\sigma)$ )。反过来, 卸载  $A$  会从  $\text{dom}(\sigma)$  中移除  $k$ , 从而破坏  $B$  的满足性; 因此, 系统会确保在  $A$  开始恢复之前,  $B$  已完全停用。于是, 依赖顺序 (依赖方在其依赖项之后激活; 依赖项在其依赖方之后卸载) 无需另行指定, 便会从通知机制中自然产生。

### 3.2.3 隔离与拦截

基本的余效应上下文  $\Sigma$  建模了一张扁平的依赖表。然而在实践中, 系统可能需要针对不同组件, 把不同的值绑定到同一个逻辑依赖。本节以两种机制扩展余效应上下文: 余效应隔离 (同一个键在不同上下文中解析为不同结果), 以及余效应拦截 (在依赖访问上施加横切行为)。

**余效应隔离。** 余效应隔离通过引入隔离域, 使同一个依赖能够在不同上下文中绑定到不同的值。它广泛适用于多租户系统、测试环境和组件沙箱。

**定义 16.** 带隔离的余效应上下文定义为

$$\Sigma^{\text{iso}} := \text{Map}(K, R) \times ((r : R) \multimap \mathcal{V}_r). \quad (12)$$

它可表示为二元组  $(\rho, \sigma)$ ，其中：

- $\rho : \text{Map}(K, R)$  是隔离域表，将逻辑键映射到域标识符；
- $\sigma : (r : R) \multimap \mathcal{V}_r$  是依赖表，即从域标识符到有类型值的依赖偏函数。

这一双层映射结构将逻辑层与存储层解耦，使依赖访问能够感知上下文。访问键  $k$  时，系统先解析  $\rho(k)$  以取得域标识符  $r$ ，再访问  $\sigma(r)$  取得实际值。

**定义 17.**  $\Sigma^{\text{iso}}$  上的  $\text{get}$ 、 $\text{set}$  和  $\text{isolate}$  操作定义如下：

$$\begin{aligned} \text{get} &: (k : K) \rightarrow \Sigma^{\text{iso}} \rightarrow \mathcal{V}_{\rho(k)}, \\ \text{get} = k &\mapsto (\rho, \sigma) \mapsto \sigma(\rho(k)), \\ \text{set} &: (k : K) \times \mathcal{V}_{\rho(k)} \rightarrow \Sigma^{\text{iso}} \rightarrow \Sigma^{\text{iso}} \times (\Sigma^{\text{iso}} \rightarrow \Sigma^{\text{iso}}), \\ \text{set} = (k, v) &\mapsto (\rho, \sigma) \mapsto \left( (\rho, \sigma[\rho(k) \mapsto v]), \lambda(\rho', \sigma').(\rho', \sigma' \setminus \rho'(k)) \right), \\ \text{isolate} &: K \times R \rightarrow \Sigma^{\text{iso}} \rightarrow \Sigma^{\text{iso}} \times (\Sigma^{\text{iso}} \rightarrow \Sigma^{\text{iso}}), \\ \text{isolate} = (k, r) &\mapsto (\rho, \sigma) \mapsto \left( (\rho[k \mapsto r], \sigma), \lambda(\rho', \sigma').(\rho' \setminus k, \sigma') \right). \end{aligned}$$

余效应隔离机制本质上实现了一个运行时特设多态系统。借助隔离域标识符，同一个依赖键可以在不同上下文中解析为完全不同的值，而且这种多态性可以在运行时动态调整。相较于传统的依赖注入，余效应隔离提供了更细粒度的控制，可以为特定组件定制隔离；所有操作仍然是效应函数 ( $\mathfrak{E}_{\Sigma^{\text{iso}}}$ )，因而继承可回退性。

**余效应拦截。** 第二种机制是余效应拦截：它把横切元数据附加到依赖访问上，从而在不修改依赖值的情况下添加行为。这些元数据既可以由上下文携带，也可以由组件声明，因此同时扩展余效应上下文和余效应规格。

**定义 18.** 带拦截的余效应上下文和规格定义为

$$\begin{aligned} \Sigma^{\text{inter}} &:= ((k : K) \rightarrow \mathcal{M}_k) \times ((k : K) \multimap (\mathcal{M}_k \rightarrow \mathcal{V}_k)), \\ \mathfrak{D}^{\text{inter}} &:= (k : K) \multimap \mathcal{M}_k. \end{aligned} \quad (13)$$

$\Sigma^{\text{inter}}$  是二元组  $(\iota, \sigma)$ ： $\iota$  是安装在上下文本身、由上下文携带的元数据，其默认值为空元数据  $\epsilon_k$ ； $\sigma$  则把每个键  $k$  映射到一个提供者函数，该函数从元数据  $\mathcal{M}_k$  映射到值  $\mathcal{V}_k$ 。规格  $d \in \mathfrak{D}^{\text{inter}}$  携带组件声明的元数据，为每个键指派其元数据  $d(k)$ ，并以  $\text{dom}(d)$  作为依赖集。每个键都为其元数据配备一个幺半群  $(\mathcal{M}_k, \oplus_k, \epsilon_k)$ ：合并操作  $\oplus_k$  满足结合律，单位元为  $\epsilon_k$ （即空元数据）。



定义 19.  $\Sigma^{\text{inter}}$  上的  $\text{get}$ 、 $\text{set}$  和  $\text{intercept}$  操作定义如下：

$$\begin{aligned}
 &\text{get} : (k : K) \times \mathcal{M}_k \rightarrow \Sigma^{\text{inter}} \rightarrow \mathcal{V}_k, \\
 &\text{get} = (k, \mu) \mapsto (\iota, \sigma) \mapsto \sigma(k)(\mu \oplus_k \iota(k)), \\
 &\text{set} : (k : K) \times (\mathcal{M}_k \rightarrow \mathcal{V}_k) \rightarrow \Sigma^{\text{inter}} \rightarrow \Sigma^{\text{inter}} \times (\Sigma^{\text{inter}} \rightarrow \Sigma^{\text{inter}}), \\
 &\text{set} = (k, \psi) \mapsto (\iota, \sigma) \mapsto \left( (\iota, \sigma[k \mapsto \psi]), \lambda(\iota', \sigma').(\iota', \sigma' \setminus k) \right), \\
 &\text{intercept} : (k : K) \times \mathcal{M}_k \rightarrow \Sigma^{\text{inter}} \rightarrow \Sigma^{\text{inter}} \times (\Sigma^{\text{inter}} \rightarrow \Sigma^{\text{inter}}), \\
 &\text{intercept} = (k, \nu) \mapsto (\iota, \sigma) \mapsto \\
 &\quad \left( (\iota[k \mapsto \iota(k) \oplus_k \nu], \sigma), \right. \\
 &\quad \left. \lambda(\iota', \sigma').(\iota'[k \mapsto \iota(k)], \sigma') \right).
 \end{aligned}$$

当规格为  $d$  的组件访问键  $k$  时，系统计算  $\sigma(k)(d(k) \oplus_k \iota(k))$ ：组件声明的元数据与上下文携带的元数据  $\iota$  合并，再将提供者函数应用于合并结果。合并遵循每个键各自的语义（例如，标量字段取右值，而集合值字段取并集），并且是右偏的，因此  $\iota(k)$  具有优先权，可以覆盖组件的声明。这样一来，外层上下文便能够约束组件使用余效应的方式，而无需修改该组件（例如第 5.2 节）。

### 3.3 组件生命周期

前两节分别定义了效应模型与余效应模型的代数结构和规格。为了赋予二者操作意义，本文将它们结合为组件，即二者共同支配其行为的运行时实体。这里的  $\Gamma$  同时充当效应上下文和余效应上下文；第 3.4 节将给出一种具体的统一构造。

定义 20. 上下文  $\Gamma$  上的组件定义为

$$\mathfrak{C}_\Gamma := \mathfrak{D}_\Gamma \times \mathfrak{E}_\Gamma, \quad (14)$$

它表示一个带有可变生命周期状态的二元组  $(d, e)$ ，其中：

- $d \in \mathfrak{D}_\Gamma$  是余效应规格，声明组件要求环境提供的依赖；
- $e \in \mathfrak{E}_\Gamma$  是效应函数，定义组件处于活跃状态时向上下文贡献的效应。

组件的两个侧面共同决定其目标状态。从效应侧看，组件必须存活，即它的效应已经应用于上下文，且对应的逆函数尚未调用；从余效应侧看，所有已声明的依赖都必须得到满足，即  $\sigma \models d$ 。当两个条件都成立时，目标状态为 **ACTIVE**；否则为 **INACTIVE**。每当目标状态发生变化时，无论变化由哪一侧引起，系统都会发起一次转换：**RELOAD** 执行组件的效应函数  $e$ ，在上下文上累积副作用；**UNLOAD** 应用累积的逆函数来恢复上下文。最简单的生命周期是图 1 所示的双状态模型。在实践中，转换会与多种控制流结构相互作用，因此还需扩展出幂等恢复、迭代、连贯性和异步语义。

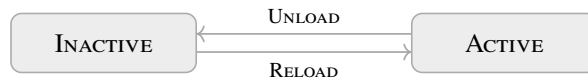


图 1 基本组件生命周期

### 3.3.1 幂等恢复

组件执行 UNLOAD 时会应用累积的逆函数来恢复上下文；为了保证这一操作安全，每个逆函数至多只能运行一次。在 recover 层面，这一性质由构造直接保证：定义 5 会把  $\varphi$  重置为  $\text{id}_\Gamma$ ，因此第二次恢复不会产生任何作用。这个义务只涉及 effect 交还给调用方的局部释放函数，也就是定义 7 中的  $\partial\Gamma \rightarrow \partial^2\Gamma$  分量；该函数会从累积的  $\varphi$  中剔除单个效应的逆函数，若调用两次，就会把该逆函数应用两次并破坏上下文。因此，本文令每个返回的释放函数均为幂等函数。

每个释放函数都以自身的私有状态保存其待命状态。这样，受防护的逆函数仍然是其所作用状态  $\gamma$  的纯函数，而不依赖隐藏的可变状态。防护的构造是生成式的 [40]：每次应用都会为新的释放函数生成一个新鲜句柄，因而任意两个释放函数都不会共享句柄，并且句柄始终封装在防护内部。实现中，每个 dispose 闭包都会新鲜捕获一个 armed 变量，以此实现上述机制。

**定义 21.** 幂等防护  $\text{idem} : (\Gamma \rightarrow \Gamma) \rightarrow (\Gamma \rightarrow \Gamma)$ （在  $\partial\Gamma$  上亦同）使一个逆函数至多在第一次调用时生效。这个包装过程是生成式的：每次应用都会生成一个新鲜句柄  $h$ ，该句柄为所生成的释放函数私有，并且不出现在 idem 的类型中。状态记录已经用过的句柄；新鲜的  $h$  不在记录中，因而仍处于有效状态，触发后则会被标记为已用：

$$\text{idem}(g) = \text{let } h \text{ fresh in } \gamma \mapsto \begin{cases} g(\gamma)[h \mapsto ()], & \text{若 } h \notin \text{dom}(\gamma), \\ \gamma, & \text{若 } h \in \text{dom}(\gamma). \end{cases} \quad (15)$$

由于  $h$  在防护构造时选定，而非由调用方提供，并且已用句柄集合只会增长，释放函数至多生效一次，同时仍保持为纯粹的带前置条件的效应函数； $\partial\Gamma$ 、 $\partial^2\Gamma$  和  $\mathfrak{F}_\Gamma$  均保持不变，已用句柄集合也不属于可回退状态。

$\text{effect}_\Gamma$ （定义 7）的幂等变体只需作一处替换：返回的逆函数  $\text{track}_\Gamma(g)$  由 idem 包装，而正向累积  $\varphi \circ g$  保持不变；后者由已经幂等的 recover 恢复。形式化地，

$$\begin{aligned} \text{effect}_\Gamma^{\text{idem}} &: \mathfrak{C}_\Gamma \rightarrow \partial\Gamma \rightarrow \partial^2\Gamma, \\ \text{effect}_\Gamma^{\text{idem}} = e &\mapsto (\gamma, \varphi) \mapsto \text{let } (\delta, g) = e(\gamma) \text{ in } ((\delta, \varphi \circ g), \text{idem}(\text{track}_\Gamma(g))). \end{aligned}$$

因为只有返回的逆函数经过了包装，所以累积函数  $\varphi$  以及第 3.1.2 节的同态结论均不受影响。

### 3.3.2 迭代

组件的 RELOAD 转换可以依次执行多个效应，而 UNLOAD 必须将它们恢复。本文用效应迭代器对多步执行建模，其中每一步都返回修改后的上下文、一个逆函数和一个延续。

**定义 22.** 效应迭代器  $\mathfrak{C}_\Gamma^{\text{iter}}$  和严格效应迭代器  $\mathfrak{C}_\Gamma^{\text{iter}*}$  定义为如下递归类型：

$$\begin{aligned} \mathfrak{C}_\Gamma^{\text{iter}} &:= \mu \mathfrak{J}. \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathfrak{J}), \\ \mathfrak{C}_\Gamma^{\text{iter}*} &:= \mu \mathfrak{J}. (e : \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathfrak{J})) \\ &\quad \times ((\gamma : \Gamma) \rightarrow (\text{let } (\delta, g, o) = e(\gamma) \text{ in } g(\delta) = \gamma)). \end{aligned} \quad (16)$$

其中， $e(\gamma)$  产生三元组  $(\delta, g, o)$ ，各分量表示：

- $\delta$  是新的上下文；
- $g$  是当前效应的逆函数；

- $o$  表示延续：
  - `Nothing` 表示迭代终止；
  - `Just( $i$ )` 给出下一次迭代步骤。

效应迭代器转换  $\text{effect}_\Gamma^{\text{iter}}$  通过递归调用，把  $\text{effect}_\Gamma$  扩展到迭代器结构上。

**定义 23.** 效应迭代器转换  $\text{effect}_\Gamma^{\text{iter}}$  定义为

$$\begin{aligned} \text{effect}_\Gamma^{\text{iter}} : \mathfrak{C}_\Gamma^{\text{iter}} &\rightarrow \partial\Gamma \rightarrow \partial^2\Gamma, \\ \text{effect}_\Gamma^{\text{iter}} = i \mapsto (\gamma, \varphi) &\mapsto \mathbf{let} (\delta, g, o) = i(\gamma) \mathbf{in} \\ &\mathbf{match} \ o \\ &| \ \mathbf{Nothing} \Rightarrow ((\delta, \varphi \circ g), \text{track}_\Gamma(g)), \\ &| \ \mathbf{Just}(i) \Rightarrow \mathbf{let} (s, r) = \text{effect}_\Gamma^{\text{iter}}(i)(\delta, \varphi \circ g) \mathbf{in} \\ &\quad (s, \text{track}_\Gamma(g) \circ r). \end{aligned}$$

在每个递归步骤中，逆函数  $g$  按应用顺序复合到  $\varphi$  上，因此累积恢复函数  $\varphi \circ g_1 \circ \dots \circ g_k$  在应用时自然会按后进先出顺序恢复效应。

在任意两个相邻步骤之间，如果目标状态已经改变，系统都可以中断转换，并应用截至当前累积的逆函数来恢复上下文。这一性质内在于迭代器模型：每一步的  $\text{Maybe}(\mathfrak{C}_\Gamma^{\text{iter}})$  延续都构成自然的中断边界，无须额外机制，如图 2 所示。从这个意义上说，效应迭代器是具体化的定义延续；主流语言通过 `yield` 运算符暴露这一结构 [41]，因此该模型可以直接映射到这些语言已经提供的生成器上。

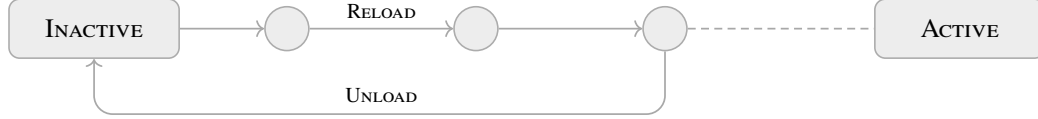


图 2 带步骤边界中断的迭代转换

单步效应  $\mathfrak{C}_\Gamma$  是一种退化情形：迭代器立即返回 `Nothing`，因而转换是原子的，不包含中断边界。对于多步转换，步骤的粒度决定了响应速度：步骤越细，系统越能迅速响应目标状态变化，但也会付出更频繁检查条件的代价。

作为进一步扩展，若把迭代器返回类型中的逆函数分量  $\Gamma \rightarrow \Gamma$  替换为另一个迭代器，就会得到双向效应迭代器

$$\mathfrak{C}_\Gamma^{\text{bidi}} = \mu\mathfrak{J}. \Gamma \rightarrow \Gamma \times \mathfrak{J} \times \text{Maybe}(\mathfrak{J}).$$

其三元组  $(\delta, p, n)$  同时给出正向延续  $n$  与反向步骤  $p$ 。这样，`UNLOAD` 也可以增量推进，并且能够被中断而转回 `RELOAD`。然而，很少有编程语言原生提供双向迭代语义，这限制了该扩展的实际适用性。

### 3.3.3 连贯性

当依赖在运行时被替换时，目标状态可能在短时间内连续经历 `ACTIVE`  $\rightarrow$  `INACTIVE`  $\rightarrow$  `ACTIVE`。如果为第一个 `ACTIVE` 发起的 `RELOAD` 仍在进行中，或者尚未中断，而第二个 `ACTIVE` 已经到来，系统就会面临依赖值已经改变的 `ACTIVE`  $\rightarrow$  `ACTIVE` 情形。若只是继续原来的 `RELOAD`，组件便会绑定到陈旧的依赖，从而破坏一致性。

为了检测这种过时状态，本文引入纪元机制，为目标状态赋予版本。对于余效应规格  $d$ ,

定义纪元函数  $\varepsilon_d : \Sigma \rightarrow E$  为

$$\varepsilon_d(\sigma) := \langle \sigma(k) \mid k \in d \rangle. \quad (17)$$

当且仅当所有依赖值都相同时，两个纪元相等：

$$\varepsilon_d(\sigma) = \varepsilon_d(\sigma') \iff \forall k \in d. \sigma(k) = \sigma'(k).$$

INACTIVE 的纪元是一个特殊常量  $\perp_E$ ，它不等于任何活跃纪元。

每次转换开始时，系统都会把目标状态的当前纪元  $\varepsilon_{\text{target}}$  记录为  $\varepsilon_{\text{inert}}$ 。在每个迭代器步骤边界，系统比较  $\varepsilon_{\text{target}}$  与  $\varepsilon_{\text{inert}}$ ：只有二者匹配时才继续下一步；否则中止转换，并应用累积的逆函数。

纪元机制实际上把生命周期从一维的 INACTIVE/ACTIVE 二元模型扩展成多维的星形结构。不同依赖配置下的 ACTIVE 构成相互独立的分支，并且都连接到 INACTIVE 状态，如图 3 所示。

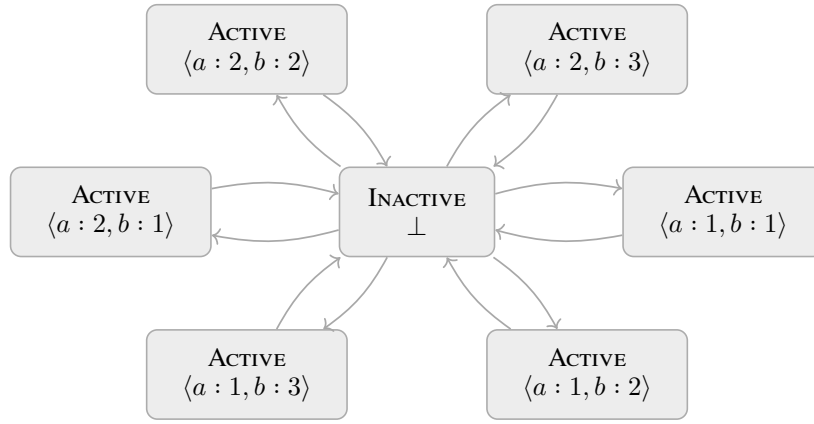


图 3 基于纪元的生命周期：每种依赖配置形成一个独立分支

### 3.3.4 异步性

同步模型假设，相对于外部状态变化，每次转换都会瞬时完成。当转换涉及异步计算时，这一假设便不再成立。本文对非即时性作抽象建模：一次转换产生一个类型为  $\text{Future}(A)$  的值，其中  $\text{Future}$  是不透明的类型构造子，其定义性质是，在提交和求值完成之间，外部状态可能发生变化。

在这一模型下，RELOAD 与 UNLOAD 会占据一段真实时间，其间目标状态可能改变。这会从两个方面威胁资源安全：立即回滚一个仍在进行的 RELOAD，会破坏尚未执行完毕的效应所要求的后进先出顺序；而快速振荡则可能导致同一个逆函数被调用两次。为了恢复安全性，本文把 RELOAD 与 UNLOAD 从瞬时转换提升为惯性状态：一旦进入某个转换，该转换便会运行至完成，之后系统才处理目标状态的任何变化。所得状态机如图 4 所示。

惯性状态的语义如下：

- 如果组件处于 RELOAD，且目标状态变为 INACTIVE，则 RELOAD 会运行至完成，随后开始 UNLOAD。如果目标状态在 RELOAD 完成前又恢复为 ACTIVE，组件便会正常进入 ACTIVE。
- 对称地，如果组件处于 UNLOAD，且目标状态变为 ACTIVE，则 UNLOAD 会先运行至完成，随后开始 RELOAD。如果目标状态又恢复为 INACTIVE，组件便会进入 INACTIVE。
- 待处理的反向转换会延迟到当前惯性状态结束；届时，系统会根据当时的目标状态执行或丢弃该转换。

迭代器步骤边界可以与惯性语义组合：在每一步，系统都会检查纪元是否仍然匹配。如果不匹配，则中止转换，应用累积的逆函数，随后开始已经延迟的转换；这正是在每个惯性转换

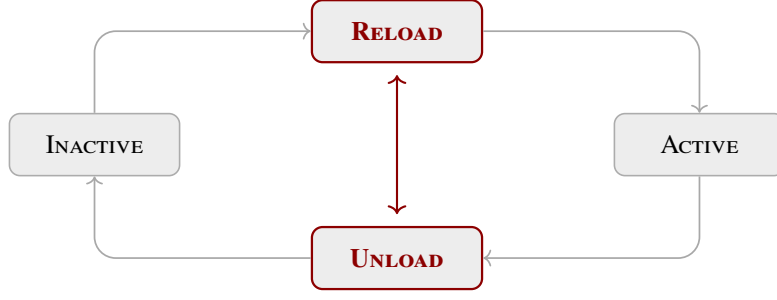


图 4 用于异步转换的惯性状态机

内部运作的同步中断机制。

### 3.4 上下文范式

第 3.3 节的组件定义使用单一上下文  $\Gamma$  同时承载效应和余效应。本节为这种统一给出一个具体构造，并论证由此得到的上下文类型本身就构成一种编程范式。

#### 3.4.1 统一上下文

对于上下文  $\Gamma$ ，效应上下文  $\partial\Gamma$ （第 3.1 节）提供了一层更高层的抽象，记录前一层上下文及该层累积的副作用。将这一结构递归化，并与余效应上下文  $\Sigma$  结合，即得到如下类型：

**定义 24.** 上下文类型  $\Gamma_\infty$  定义为

$$\Gamma_\infty := \mu\Gamma. \Gamma \times (\Gamma \rightarrow \Gamma) \times \Sigma, \quad (18)$$

其中三个投影分别为：

- $\Gamma$ ：当前上下文状态（递归）；
- $\Gamma \rightarrow \Gamma$ ：用于效应恢复的累积逆函数；
- $\Sigma$ ：承载依赖信息的余效应上下文。

根据这一定义，效应成为  $\Gamma_\infty$  上的自同态，从而将  $\partial$  塔统一成单一的自相似类型。余效应上下文  $\Sigma$  在结构上融入其中：依赖操作（set、get）作用于  $\Sigma$  分量，而第二个分量中的逆函数跟踪这些操作的逆转。由于作为  $\Sigma$  基础的类型族  $\mathcal{V}$  不受约束，系统需要在组件间共享的任意全局状态，都可以编码成具有适当值类型的依赖项； $\Sigma$  因而涵盖所有共享可变状态，而不只是组件间依赖。组件与其环境之间的每一次交互，都经由这一单一实体完成。

**层次化组合。**  $\Gamma_\infty$  的递归结构支持层次化控制：父上下文聚合多个子层级的效应，形成树状控制结构；该结构既保持模块化，又支持跨层级的统一管理。效应变换将“插接”这一隐喻直接落实为：

- 加载组件对应于执行其效应（插入）；
- 卸载组件对应于恢复其效应（拔出，且不影响其他正在运行的组件）；
- 层次结构中不同层级的组件可以各自独立加载和卸载；父上下文聚合并管理其所有子组件的效应，从而支持任意深度的嵌套组合。

**指称与实现。** 将一个操作定型为  $\Gamma_\infty$  上的效应，会固定其指称，即一个后继状态及其对应的逆函数，但不会固定其实现；实现决定该逆函数如何执行。

**定义 25.** 效应函数  $f : \mathfrak{E}_{\Gamma_\infty}$  允许两种实现：



- 原地实现会修改上下文，并返回一个非平凡的逆函数；后继状态是输入的别名，而恢复过程通过运行逆函数来逆转该修改。
- 派生实现保持输入不变，在  $\Gamma_\infty$  的递归结构中返回一个新的上下文，并以恒等函数作为其逆函数；恢复过程则丢弃该派生上下文。

在纯函数式环境中，这两种实现彼此重合；命令式宿主则可以逐个操作选择其中任何一种。第 4.1.2 节将给出两种实现的代表性实现方式。

### 3.4.2 上下文范式的定位

各种编程范式在处理副作用的方式上存在根本差异。两种既有范式构成这一谱系的两极：

**显式状态传递（函数式）。** 为了保持引用透明性，纯函数式语言将副作用建模为对状态的显式变换。状态单子  $S \rightarrow (A, S)$  [22] 让环境贯穿每一次计算。这种方式提供了强有力的组合性保证：效应在类型中可见，并且可以进行等式推理。然而，它也带来了显著的人体工学成本：调用链中的每个函数都必须接收并返回状态参数，即使该函数只是将状态原样向下传递。当效应维度增加时（如日志、配置和 I/O），单子堆叠或效应处理器的样板代码便会迅速增多。

**隐式变更（命令式/OOP）。** 主流命令式语言允许组件修改共享状态并访问依赖，而无须在调用点显式声明。在效应一侧，React 的 `useEffect` 钩子是一个典型例子：它在组件内部的线程上注册持久副作用，但效应目标和注册机制都不作为显式参数出现，其标识依赖于隐藏运行时状态中的调用顺序位置。在余效应一侧，Java 的服务定位器模式（例如 Spring 的 `ApplicationContext.getBean(...)`）在运行时从全局注册表中取得依赖，要求每个调用点都执行空值检查和类型转换；依赖关系则隐含并散落于整个代码库。更一般地说，要理解 `f()` 如何修改系统或依赖系统，就必须沿调用关系递归阅读其实现。重构也因此变得脆弱，因为移动或删除某次调用，都可能悄无声息地破坏远处的不变量。

上下文范式将函数式方法的可追溯性与命令式方法的人体工学优势结合起来。效应和余效应都通过显式的上下文参数来介导。因此，每个操作都可以归属于它所调用的特定上下文，进而归属于该上下文所属的组件。

上下文范式不仅结合了这两极的优势，还允许开发者分别处理每个效应和依赖项，并自动将它们组合为系统行为。对于可回退效应，开发者提供每个原子操作的逆函数，任意复合操作的逆函数则由组合自然得到（第 3.1 节），因此加载和拆卸从构造上就是可逆的。对于反应式余效应，组件只需声明其所需的依赖项，运行时便会自动解析并重新连接这些依赖（第 3.2 节），使其在提供者被添加、移除或替换时始终保持正确连接。在这两个方向上，原本需要依赖开发者自律来保证的正确性，都成为该范式的一种结构性属性。

## 4 实现与案例研究

本节介绍 Cordis，它将第 3 节的形式模型实现为一种实用的编程抽象。Cordis 是一个面向时空可组合性的元框架：不同于针对特定领域的应用框架（例如 Web 路由、ORM 和 UI 渲染），它不预设任何具体场景，唯一职责是提供通用的动态组合语义。其实现分为三个层次：（1）核心库（第 4.1 节）直接实现效应系统与余效应系统；（2）组件加载器（第 4.2 节）在核心库之上扩展配置协调和热模块替换能力；（3）Koishi 等应用框架（第 4.3 节）在前两个层次之上构建领域特定功能。



| 理论 (第 3 节)                                               | 实现                                           |
|----------------------------------------------------------|----------------------------------------------|
| $\Gamma_\infty$                                          | ctx, 一等上下文                                   |
| $\mathfrak{C}_\Gamma, \mathfrak{C}_\Gamma^{\text{iter}}$ | 返回或逐次产出逆变换的效应回调                              |
| $\text{effect}_\Gamma^{\text{iter}}(e)$                  | ctx.effect(callback)                         |
| $\Sigma, \Sigma^{\text{iso}}, \Sigma^{\text{inter}}$     | ctx[@@store]、ctx[@@isolate]、ctx[@@intercept] |
| $\text{get}(k), \text{set}(k, v)$                        | ctx.get(key)、ctx.set(key, value)             |
| $\text{isolate}(k, r)$                                   | ctx.isolate(key, realm)                      |
| $\text{intercept}(k, \nu)$                               | ctx.intercept(key, metadata)                 |
| $\mathfrak{C}_\Gamma$                                    | fiber, 组件的运行时实例                              |
| $d : \mathfrak{D}_\Gamma$                                | fiber.inject                                 |
| $e : \mathfrak{C}_\Gamma$                                | fiber.apply                                  |
| $\varepsilon_d(\sigma)$                                  | fiber.epoch                                  |
| recover                                                  | fiber.dispose, 累积的逆变换                        |

表 1 理论与实现的对应关系

## 4.1 核心库

表 1 汇总了理论构造与运行时对应项之间的关系。具体而言, 本节以下统一使用后文引入的运行时名称, 仅在形式对应中保留理论符号。另以 @@name 表示框架内部的符号键, 因此 ctx[@@store] 中的方括号表示按符号键访问上下文中的不透明槽位, 而不是在以字符串为键的映射中进行索引。

线程 (fiber) 是组件的运行时实例: 它是一个有状态对象, 负责承载组件经历第 3.3 节所述的生命周期, 并将组件的静态规格与生命周期算法读取的实时转换状态封装在一起。静态规格包括余效应规格 fiber.inject ( $d$ ) 和效应函数 fiber.apply ( $e$ ), 以及两个相关上下文: fiber.parent 是实例化该线程所依托的上下文, fiber.ctx 则是由前者派生 (定义 25)、供该线程在其中运行的全新子上下文。转换状态包括累积逆变换 fiber.dispose (recover)、目标状态版本 fiber.epoch ( $\varepsilon_d(\sigma)$ , 其中  $\perp$  表示 INACTIVE), 以及正在进行的转换句柄 fiber.inertia。表 1 将这些运行时表示与其理论对应项逐一列出。

本节余下部分自底向上构建核心库。第 4.1.1 节实现可回退效应, 它是修改上下文的唯一原语; 第 4.1.2 节在其上实现反应式余效应; 第 4.1.3 节将二者组合为组件生命周期; 第 4.1.4 节则公开建立在这些机制之上的上下文级操作。

### 4.1.1 效应跟踪

本节实现可回退效应 (第 3.1 节)。Cordis 中的每一次上下文变更都经由同一个原语 ctx.effect: 余效应供给、组件实例化以及其他所有修改上下文的操作, 最终都归约为一次 ctx.effect 调用。因此, 通过上下文执行的任何操作都会被自动跟踪, 并在组件卸载时得到恢复。

从操作层面看, ctx.effect 实现了  $\text{effect}_\Gamma^{\text{iter}}$  (定义 23): 它接收一个类型为  $\mathfrak{C}_\Gamma^{\text{iter}}$  的回调, 并将其提升为  $\mathfrak{C}_{\text{of}}^{\text{iter}}$ , 由此产生一个 dispose 闭包; 调用该闭包即可恢复相应效应。Cordis 通过这一个操作同时接受  $\mathfrak{C}_\Gamma$  和  $\mathfrak{C}_\Gamma^{\text{iter}}$  (特设多态); 此处以迭代器形式为代表, 因为普通效应函数只是仅产出一个逆变换的退化迭代器。

算法 1 给出了 ctx.effect 的构造。以  $f \circ g$  表示先运行  $g$ 、再运行  $f$  的释放函数, 以 id 表示空操作; 因此, 将每个新的逆变换前置便会得到后进先出 (LIFO) 的恢复顺序。

执行引擎 execute 将回调作为效应迭代器 ( $\mathfrak{C}_\Gamma^{\text{iter}}$ , 定义 22) 驱动, 并把每一步产出的逆变换折叠为一个复合逆变换。它会在每一步之前检查调用方提供的 guard; 一旦守卫条件失效, 迭代便会停止, 仅保留截至当时已累积的逆变换。这正是第 3.3.2 节所述的步边界中断: 每一步的 Maybe( $\mathfrak{C}_\Gamma^{\text{iter}}$ ) 续延由迭代器的 done 标志与 guard 共同实现。

**算法 1** 效应跟踪

---

```

1 async function execute(callback, guard)
2   iter ← callback()
3   inverse ← id
4   while guard()
5     (value, done) ← await iter.next()
6     if value then inverse ← value ◦ inverse
7     if done then break
8   return inverse
9 function effect(ctx, callback)
10  armed ← true
11  task ← execute(callback, () ↦ armed)
12  async function dispose()
13    if not armed then return
14    armed ← false
15    recover ← await task
16    recover()
17  ctx.dispose ← dispose ◦ ctx.dispose
18  return dispose

```

---

`ctx.effect` 是 `execute` 之上的一层轻量封装，额外提供两项能力。第一，幂等的自我释放（实现定义 21 中的 `idem`）：守卫返回 `armed` 标志，而返回的 `dispose` 会将 `armed` 置为 `false`，从而同时终止任何尚在进行的迭代，并确保恢复至多触发一次。第二，父上下文组合：`dispose` 被前置到外层上下文的累积逆变换 `ctx.dispose` 中，因此子效应的逆变换本身也是父上下文上的一个效应，从而实现  $\partial^2\Gamma$  的递归结构。组件层（第 4.1.3 节）复用同一个 `execute`，但其守卫检查的是纪元稳定性，而不是 `armed`。

**4.1.2 余效应操作**

本节给出反应式余效应（第 3.2 节）的具体实现。所有余效应操作都作用于每个上下文携带的三个以符号为键的槽位：

- `@@store`：值存储  $\sigma : (r : R) \rightarrow \mathcal{V}_r$ ，将域符号映射到有类型的值；
- `@@isolate`：域表  $\rho : \text{Map}(K, R)$ ，将余效应键映射到域符号；
- `@@intercept`：拦截表  $\iota : (k : K) \rightarrow \mathcal{M}_k$ ，为每个键指定其元数据。

前两个槽位组合成双层解析  $k \rightarrow \rho(k) \rightarrow \sigma(\rho(k))$ ：`ctx.get(key)`（算法 2）先从 `@@isolate` 读取域符号  $\rho(k)$ ，再从 `@@store` 读取已绑定的值  $\sigma(\rho(k))$ 。通过  $\rho$  这一间接层，隔离操作可以把一个键重定向到独立的绑定；`@@intercept` 则仅在访问绑定时才会被查阅，它调整的是绑定的使用方式，而非绑定解析到的内容。这些操作分两部分实现：（1）*提供与通知*，负责建立或撤销绑定，并将变化传播给依赖方；（2）*隔离与拦截*，负责重塑键的解析方式。

**提供与通知。** 由于  $\text{set}(k, v)$  的类型为  $\mathfrak{C}_\Sigma$ （第 3.1 节），提供余效应就是一次 `ctx.effect` 调用，因而继承其自动跟踪与恢复机制。算法 2 实现了 `ctx.set(key, value)`，即具体的  $\text{set}(k, v)$ ：回调函数以域符号  $\rho(k)$  为键，将值绑定到存储中；返回的 `dispose` 函数则移除该值。建立和移除绑定时都会调用 `notify`，将变化传播给依赖该绑定的组件。

算法 3 将每次绑定变化传播给依赖方：对于每个存活纤维，它检查发生变化的键是否出现在该纤维的 `fiber.inject` 中，并且是否解析到同一个域；若是，则调用 `refresh`（第 4.1.3 节），根据新状态重新求值该纤维。这对应定义 15 中的反应式分类：当一次变化使  $\sigma \models d$  的真值发

**算法 2** 余效应操作

---

```

1 function get(ctx, key)
2   realm  $\leftarrow$  ctx[@@isolate][key]  $\triangleright \rho(k)$ 
3   return ctx[@@store][realm]  $\triangleright \sigma(\rho(k))$ 
4 function set(ctx, key, value)
5   function callback()
6     realm  $\leftarrow$  ctx[@@isolate][key]  $\triangleright \rho(k)$ 
7     ctx[@@store][realm]  $\leftarrow$  value  $\triangleright \sigma[\rho(k) \mapsto v]$ 
8     notify(ctx, [key])
9     return function()
10      delete ctx[@@store][realm]  $\triangleright \sigma \setminus \rho(k)$ 
11      notify(ctx, [key])
12   return ctx.effect(callback)

```

---

**算法 3** 反应式通知

---

```

1 function notify(ctx, keys)
2   for fiber in all_fibers do
3     for key in keys do
4       if key  $\in$  fiber.inject and
5         fiber.ctx[@@isolate][key] =
6           ctx[@@isolate][key] then
7         refresh(fiber)
8     break

```

---

生改变时，纤程会被激活或停用；而 `refresh` 的幂等性则使中性变化不会产生影响。这种重新求值与各种控制流之间的交互将在第 4.1.3 节中展开。

**隔离与拦截。** 这两种操作在结构上执行同一类动作：它们各自派生一个子上下文，针对指定的 `key` 调整一张继承而来的表，同时保持父上下文不变。因此，恢复是隐式的：只需丢弃子上下文，无须执行显式逆变换。`ctx.isolate(key, realm)` 使用 `realm` 覆盖域映射  $\rho$ ；若未指定 `realm`，则默认使用一个新生成的符号。该操作实现了定义 17 中的 `isolate`，因此，同一键在两个被指派不同符号的上下文中会解析到彼此独立的绑定。`ctx.intercept(key, metadata)` 将 `metadata` 合并到拦截表  $\iota$  中，实现了定义 19 中的 `intercept`；按照该定义，新元数据会与上下文中该键已有的元数据合并，并优先于后者。

**4.1.3 组件生命周期**

组件由 `ctx.use` 实例化为纤程。本节赋予纤程（于第 4.1 节引入）以第 3.3.4 节惯性状态机的操作语义。下述算法由两个字段驱动：`fiber.parent` 是 `fiber.ctx` 的父上下文，由此形成组件层级（即第 3.4.1 节中  $\Gamma_\infty$  的递归结构）；`fiber.inertia` 是指向正在进行的异步迁移的句柄（若处于空闲状态则为 `null`）。

算法 4 展示组件实例化。一个组件将余效应规格 `component.inject(d)` 与效应函数 `component.apply` 配成一对；实例化过程将组件的 `config` 绑定至 `fiber.apply`（第 9 行），后者是已应用配置的效应函数（ $e$ ），随后由生命周期执行。回调函数（第 2 行）是在父纤程中跟踪的效应：执行时，它调用 `refresh`（算法 5）以启动子纤程的生命周期；恢复时，它强制将子纤程的纪元设为  $\perp$  并触发 `unload`。这确保卸载父组件会级联至所有子组件，直接体现了父级效应上下文上的  $\diamond$  组合。

算法 5 实现了第 3.3.4 节的惯性状态机，其中 `RELOAD` 和 `UNLOAD` 都具有惯性：一旦进入某

**算法 4 组件实例化**


---

```

1 function use(ctx, component, config)
2   function callback()
3     refresh(fiber)
4   return function()
5     fiber.epoch  $\leftarrow \perp$ 
6     unload(fiber)
7   fiber  $\leftarrow$  Fiber(parent: ctx, inject: component.inject)
8   fiber.ctx  $\leftarrow$  ctx[fiber  $\mapsto$  fiber]
9   fiber.apply  $\leftarrow$  ()  $\mapsto$  component.apply(fiber.ctx, config)
10  ctx.effect(callback)
11  return fiber

```

---

个迁移，系统便先让该迁移运行至完成，然后才响应目标状态的变化。`refresh` 函数根据余效应存储重新计算纤维的纪元；如果该纤维尚未处于迁移中，就启动 `reload` 或 `unload` 任务<sup>2</sup>。`reload` 函数记录当前纪元，并执行组件的效应函数 `apply`。执行完成后，它检查纪元是否仍然匹配：若匹配，纤维稳定于 `ACTIVE`；若不匹配（无论新纪元为  $\perp$ ，还是另一个活跃配置），则串联进入 `unload`。与之对称，`unload` 按后进先出顺序恢复所有已跟踪的效应，随后稳定于 `INACTIVE`，或串联进入 `reload`。这种相互递归实现了惯性属性：一个迁移一旦开始，就会先完成，然后才允许任何新的迁移启动。

纪元的计算方式是：针对当前余效应存储解析每个已声明的键，再将所得值组成元组（第 3.3.3 节）。由于 `notify`（第 4.1.2 节）会在每次余效应变化时重新计算纪元，因此纤维恰好在其解析值发生变化时重载。

该算法在两个互补的层次上运行。在迁移层次，`reload` 与 `unload` 会在迁移完成时检查纪元，从而支持跨迁移的惯性串联。在每次迁移内部的步骤层次，效应执行过程（算法 1）会在每个迭代器步骤的边界检查纪元，从而支持单次迁移内的部分回滚。这两种机制分别对应第 3.3.3 节的迁移间纪元检查与迁移内纪元检查。

**4.1.4 上下文访问**

第 4.1.2 节的余效应操作构成了一套反射式 API：余效应通过 `ctx.set(key, value)` 写入，并通过 `ctx.get(key)` 读取，两者都以名称为键。在这套反射式 API 之上，Cordis 又提供了第二种更贴近宿主语言的上下文扩展与使用方式：属性访问。组件可以像访问上下文的原生结构一样，将余效应作为属性 `ctx[key]` 访问，而无须调用方法。在 TypeScript 中，Cordis 通过 Proxy 实现这一机制，由其 `get` 捕获器对每次属性访问进行中介。算法 6 展示上下文如何在第 4.1.2 节原语 `get` 之上，将这类访问解析为余效应。

算法 6 从发起访问的上下文开始，沿纤维链向上遍历：遇到第一个在其 `inject` 中声明了 `key` 的纤维时，访问即获授权，并通过 `get`（算法 2）取得相应值；若遍历到根节点仍未发现这样的声明，则拒绝访问。这正是代理与直接调用 `ctx.get` 的区别所在：`ctx.get(key)` 是全定义的查找操作，它返回已绑定的值或空值，且永不失败；代理则拒绝访问任何未声明的余效应，从而在使用点强制执行余效应规格  $d$ 。此处不会出现“已声明但不存在”的值，因为只有在所有已声明余效应均已满足后，纤维才会进入 `ACTIVE`（第 4.1.3 节）。

这种拒绝是在访问点执行的运行时检查。由于组件的余效应规格  $d$  是静态声明的，原则

---

<sup>2</sup>`create_task` 会调度一个异步函数并发运行，并返回该函数的句柄（存储在 `fiber.inertia` 中）。为保持语言无关性，算法显式写出这一操作：在采用急切调度的语言机制中（例如 TypeScript 的 `Promise`），调用是隐式的，返回的 `Promise` 即为句柄；在采用惰性调度的语言机制中（例如 Python 的协程、Rust 的 `Future`），宿主必须生成该任务，任务才会推进。

**算法 5** 组件生命周期

---

```

1 function refresh(fiber)
2    $epoch \leftarrow \varepsilon_d(\sigma)$ 
3   if  $epoch = \text{fiber.epoch}$  then return
4    $\text{fiber.epoch} \leftarrow epoch$ 
5   if  $\text{fiber.inertia}$  then return
6   if  $epoch \neq \perp$  then
7      $\text{fiber.inertia} \leftarrow \text{create\_task}(\text{reload}(\text{fiber}))$ 
8   else
9      $\text{fiber.inertia} \leftarrow \text{create\_task}(\text{unload}(\text{fiber}))$ 
10 async function reload(fiber)
11    $epoch_0 \leftarrow \text{fiber.epoch}$ 
12    $\text{recover} \leftarrow \text{await execute}(\text{fiber.apply}, () \mapsto \text{fiber.epoch} = epoch_0)$ 
13    $\text{fiber.dispose} \leftarrow \text{recover} \circ \text{fiber.dispose}$ 
14   if  $\text{fiber.epoch} = epoch_0$  then
15      $\text{fiber.inertia} \leftarrow \text{null}$ 
16   else
17      $\text{fiber.inertia} \leftarrow \text{create\_task}(\text{unload}(\text{fiber}))$ 
18 async function unload(fiber)
19   await  $\text{fiber.dispose}()$ 
20    $\text{fiber.dispose} \leftarrow \text{id}$ 
21   if  $\text{fiber.epoch} = \perp$  then
22      $\text{fiber.inertia} \leftarrow \text{null}$ 
23   else
24      $\text{fiber.inertia} \leftarrow \text{create\_task}(\text{reload}(\text{fiber}))$ 

```

---

**算法 6** 由 Proxy 介导的上下文访问

---

```

1 function resolve(ctx, key)
2    $\text{fiber} \leftarrow \text{ctx.fiber}$ 
3   repeat
4     if  $\text{key} \in \text{fiber.inject}$  then return  $\text{get}(\text{ctx}, \text{key})$ 
5     if  $\text{fiber} = \text{root}$  then throw UNDECLARED_ACCESS
6      $\text{fiber} \leftarrow \text{fiber.parent.fiber}$ 

```

---



上也可以在编译期检测同一违规：执行前，根据已声明的  $d$  解析每个 `ctx[key]` 即可。第 5.3 节将讨论宿主语言如何借助类型层级的依赖声明与编译期元编程，实现完全相同的中介。

## 4.2 组件加载器

核心库为组件开发者提供用于动态组合的命令式原语，例如 `ctx.effect`、`ctx.use` 和 `ctx.set`。应用编排器则面临另一项独立的问题：它们需要把既有组件组装为一个运行中的系统，并在系统的整个生命周期内调整其组合。组件加载器通过引入声明式配置层来处理这一问题：编排器以持久数据结构指定所需的组合，加载器则把这项规格的变更转译为相应的命令式线程操作。

### 4.2.1 声明式配置层

Cordis 系统的运行时由若干线程组合而成；这些线程依照自身所处的上下文层级形成一棵树（第 3.4.1 节）。通过把每个线程序列化为一个条目，并把这些条目组织成相同的树形结构，声明式配置层便将应用的结构捕获为一种持久规格。

**定义 26.** 一个条目声明一个线程，并记录以下字段：

- `id`：稳定标识符；当所属群组的子条目列表发生变化时，它用作协调键；
- `url`：待实例化组件模块的 URL；
- `isolate`：施加到该条目上下文上的隔离标注；
- `intercept`：施加到该条目上下文上的拦截标注；
- `config`：绑定到组件中的配置，用于形成组件的效应函数 `apply`；
- `disabled`：该条目是否已被管理性禁用。

在运行时，一个条目管理其所声明的线程，并响应这些字段的变化，由此将声明式规格绑定到它的命令式实现。

这些条目构成一棵配置树，作为系统所加载内容的权威记录。一个条目可以是映射到单个线程的叶节点；它的组件也可以继续加载更多组件，使该条目成为分支节点。Cordis 为这种分组与嵌套加载提供了专用组件：`@cordisjs/group` 接受一个子条目列表作为配置，并将其加载为一个子群组；`@cordisjs/include` 则加载外部配置文件（YAML 或 JSON），并把文件中的条目嫁接为一棵嵌套子树。

当条目记录发生变化时，加载器会进行增量协调：它不会拆除整个线程再从头重建，而是检查发生变化的字段，并针对每个字段执行干扰最小的操作。

- `id`、`url`：重建该条目，因为它的身份或组件已经变化；
- `isolate`：重写该条目的域映射表，迁移该条目所提供的所有余效应，并通知解析结果发生变化的依赖方；
- `intercept`：原地更新；拦截元数据是在读取时查询的，无需重新加载；
- `config`：交给组件，由组件决定如何应用新的载荷；典型做法是将其与先前载荷比较，仅在发生实质变化时重新加载。特别地，`@cordisjs/group` 条目的 `config` 是它的子条目列表，因此它以子条目 `id` 为键作差分，分别创建、移除或更新各个子条目。由于更新仍然存在的子条目会再次进入同一套按字段分派的过程，群组协调与条目更新便一同沿树向下递归；
- `disabled`：设为真时卸载线程，清除时重新加载线程。

Cordis 还允许组件在运行时更新自身的 `config`，或自行禁用。无论哪种情况，加载器都会把变更写回配置层，使持久化规格始终忠实反映正在运行的系统。

### 4.2.2 热模块替换

热模块替换 (HMR) 把可回退效应模式应用到模块层：当源文件发生变化时（通常是在开发期间），系统会原地替换受影响的模块，而不重新启动进程。由于纤程已经为其组件的所有效应和余效应划定了边界，本身即为组件的模块只需通过纤程操作便可替换：释放旧纤程会恢复组件所安装的一切，而从已重新加载的模块实例化一个新纤程则会将其重新安装。因此，与 Webpack[42] 或 Vite[43] 的 HMR 不同，Cordis 的 HMR 不需要由开发者标注接受边界。

@cordisjs/hmr 组件提供 HMR 引擎，其运行分为三个阶段。

**阶段 1：模块分类。** 引擎接受两个输入：暂存集（自上次重新加载以来内容已发生变化的文件 URL）与外部模块集（无法热替换、因而会触发完整重启的模块）。用 `get_imports(url)` 表示 `url` 直接导入的模块，引擎会对这些变更所涉及的依赖子图进行分类，把每个模块标记为已接受或已拒绝：

---

#### 算法 7 模块分类

---

```

1 function classify(stashed, externals)
2   accepted  $\leftarrow$  stashed
3   declined  $\leftarrow$  externals
4   pending  $\leftarrow$   $\emptyset$ 
5   for url in stashed do
6     pending  $\leftarrow$  pending  $\cup$  (get_imports(url)  $\setminus$  (accepted  $\cup$  declined))
7   repeat
8     progress  $\leftarrow$  false
9     for url in pending do
10      if get_imports(url)  $\cap$  accepted  $\neq \emptyset$  then
11        accepted  $\leftarrow$  accepted  $\cup$  {url}
12        pending  $\leftarrow$  pending  $\setminus$  {url}
13        progress  $\leftarrow$  true
14      else if get_imports(url)  $\subseteq$  declined then
15        declined  $\leftarrow$  declined  $\cup$  {url}
16        pending  $\leftarrow$  pending  $\setminus$  {url}
17        progress  $\leftarrow$  true
18      else
19        pending  $\leftarrow$  pending  $\cup$  (get_imports(url)  $\setminus$  (accepted  $\cup$  declined))
20    until not progress
21    declined  $\leftarrow$  declined  $\cup$  pending
22  return (accepted, declined)

```

---

固定点计算以暂存文件的导入为种子：只要某个模块的任一导入已被接受，该模块便被接受；一旦它的所有导入均被拒绝，该模块便被拒绝；任何仍处于未决状态、陷在导入环中的模块，默认归入已拒绝集合。

**阶段 2：失效条目检测。** 引擎随后使用 `accepted` 和 `declined` 筛选组件条目，只保留依赖树可达已变更模块的失效条目。它使用 `get_dependencies` 遍历每个条目的依赖树；该函数收集一个模块的传递导入，并把 `declined` 当作遍历边界：

一个条目恰在其依赖树与 `accepted` 相交时失效；随后，这棵树会被并入 `accepted`，从而使沿途的每个失效模块都在下一阶段被作废。

**算法 8** 失效条目检测

---

```

1 function get_dependencies(root, declined)
2   deps  $\leftarrow \emptyset$ 
3   function traverse(url)
4     if url  $\in$  deps or url  $\in$  declined then return
5     deps  $\leftarrow$  deps  $\cup$  {url}
6     for child in get_imports(url) do traverse(child)
7   traverse(root)
8   return deps
9 function detect(entries, accepted, declined)
10  stale_entries  $\leftarrow \emptyset$ 
11  for entry in entries do
12    tree  $\leftarrow$  get_dependencies(entry.url, declined)
13    if tree  $\cap$  accepted  $\neq \emptyset$  then
14      accepted  $\leftarrow$  accepted  $\cup$  tree
15      stale_entries  $\leftarrow$  stale_entries  $\cup$  {entry}
16  return stale_entries

```

---

**阶段 3：事务性重载。** 最后，引擎重新加载失效条目。它先使已接受模块的缓存失效<sup>3</sup>，并备份每个被移除的模块以支持回滚；随后按各自的 *url* 重新导入每个失效条目的组件模块，并换入一个全新的线程：

**算法 9** 事务性模块重载

---

```

1 function reload(ctx, accepted, stale_entries)
2   backup  $\leftarrow$  invalidate_caches(accepted)
3   try
4     for entry in stale_entries do
5       entry.fiber.dispose()
6       entry.fiber  $\leftarrow$  ctx.use(import(entry.url), entry.config)
7   catch error
8     restore_caches(backup)
9     for entry in stale_entries do
10      entry.fiber.dispose()
11      entry.fiber  $\leftarrow$  ctx.use(backup[entry.url], entry.config)
12   throw error

```

---

事务性保证确保系统绝不会进入只完成一半的重载状态：若任一模块导入失败（例如出现语法错误），系统就会恢复缓存，并从 *backup*[*entry.url*] 重建每个失效条目；该值正是缓存刚刚得到恢复的先前组件，由此撤销所有已经完成的交换。

**4.3 案例研究：Koishi**

Koishi 是一个构建于 Cordis 之上的开源聊天机器人应用框架<sup>4</sup>。经过四年多的开发，它已经积累了 4000 多个由社区贡献的插件<sup>5</sup>，涵盖即时通信（IM）适配器、数据库驱动程序、管

<sup>3</sup>在 Node.js 上，这意味着同时清除 ES 模块系统和 CommonJS 模块系统的缓存，因为经由 ES 加载器导入的模块可能同时出现在二者之中。

<sup>4</sup>Koishi 当前使用 Cordis v3。本文介绍 Cordis v4，它细化了效应与余效应的语义，并重新设计了加载器；两个版本共享核心组合模型。

<sup>5</sup>Koishi 使用 *plugin*（插件）一词指称本文形式化为 *component*（组件）的概念。

理控制台和面向最终用户的功能。其规模和多样性使之成为在生产环境中验证 Cordis 动态可组合性的代表性案例。

**元框架的表达能力和通用性。** Koishi 作为服务端机器人运行，其所有能力都是在第 4.1 节的上下文原语之上以插件形式实现的；Koishi 本身只提供聊天机器人领域的词汇。相同的模型也出现在一个全然不同的运行时中：Koishi 的 Web 控制台是另一个独立的 Cordis 应用，它的插件组合的是浏览器及其用户界面的原语，而不是服务器的原语。上述迥异的场景印证了第 3 节模型的两个属性。(1) 该模型具有表达能力：其原语足以承载一个完整的生产系统，宿主框架只需提供领域词汇。(2) 该模型具有通用性：它规定效应与余效应如何组合，同时将二者的含义留给各个应用决定，因此既不预设特定领域，也不预设特定运行时。

**无需认知开销的时间可组合性。** 第 1.2.1 节考察的插件系统无法在不重启扩展宿主的情况下卸载单个扩展的效应。Koishi 则经常执行这一操作：编排器从控制台停用某个插件，其效应随即在原处撤销；在开发期间，HMR 引擎会在保存时重新应用编辑过的插件，同时保留系统其他部分的缓存状态与活动连接。Cordis 不仅使这种移除成为可能，还让插件作者几乎无须为此付出额外工作。由于通过上下文实施的效应会被自动跟踪，其逆函数也会自动组合（第 3.1 节），即使经验不足的作者没有编写卸载路径，也能让插件中经由上下文实施的效应按顺序得到清理。这实现了第 1.2.1 节指出其缺失的关注点局部性：原本要靠每位作者勤勉才能满足的正确性要求，转而由这一抽象一次性统一履行。

**跨开放生态系统的空间可组合性。** 第 1.2.1 节所述插件系统大多缺乏插件间依赖；相比之下，Koishi 生态呈现出真正的依赖拓扑：即时通信适配器提供对各个消息平台的访问，数据库驱动程序提供持久化存储，功能插件则将这些能力声明为余效应并加以访问。在运行时重新配置提供者，例如切换存储后端或重新连接适配器，只会重新激活那些解析所得依赖发生变化的依赖方（第 3.2 节）；依赖项不可用的插件会保持非活跃状态，直至该依赖项出现，而不会报错。该案例研究所证实的是，这种组合能够跨越独立编写的代码而成立：插件与其依赖项通常由不同作者编写，作者之间除连接二者的余效应外，无须协调任何事项；因此，反应式余效应可以让这一组合体在由独立贡献者构成的开放生态系统中保持一致。

**效度威胁。** 本文证据取自单一宿主语言中的单一生态系统，因此无法将该范式本身的优点同其 TypeScript 实现或 Koishi 特定领域的优点区分开来；这些证据也来自观察，而不是同替代架构进行的受控比较。因此，该案例研究证明的是这一范式存在且已被采用，而不是给出定量结果；以某个基线为参照，衡量这一抽象的开销及其对开发者生产力的影响，仍有待未来工作完成。

## 5 讨论

前文给出的形式模型与实现引入了一种面向动态可组合性的编程范式。本节考察这一范式如何扩展到更广泛的工程问题，并讨论其中的设计张力与开放问题。

### 5.1 服务多路复用

OSGi 等动态组件平台 [44] 围绕服务来组织组合：服务是提供者以某个接口发布、消费者加以绑定的能力单元。Cordis 的余效应模型与这一概念相呼应，其中的服务对应于某个键背后的接口。提供服务的组件是该服务的提供者，注入服务的组件则是其消费者。同一项服务可以由多个提供者实现，而这种多重性可以通过两种形式实现。(1) 独占绑定：多个实现共享同一



接口，但任一时刻至多绑定一个实现；编排器选择所绑定的实现，而在实现之间切换时，需要卸载一个提供者并加载另一个提供者，这会短暂扰动每个消费者的依赖。(2) *服务代理*：一个充当该接口入口点的中央服务，同时作为依赖被后端提供者和消费者注入；因此，多个提供者可以共存，代理则在它们之间派发每个请求。与独占绑定相比，代理会吸收这种扰动：更新后端提供者时，代理仍留在原位，所以消费者看不到其依赖发生变化，也不会触发重新加载。

服务代理是三种能力的基础：负载均衡、滚动更新与跨进程调用。

**负载均衡。** 当多个提供者共存时，代理按照可配置的策略（例如轮询、最少负载或延迟加权）在它们之间分发请求，或者按消费者点名的显式目标进行分发。由于提供者都是普通组件，因此可以通过添加或移除提供者来扩大或缩小容量；每个提供者都通过一个可回退效应向代理注册，所以卸载提供者会回退这项注册，并自动将其从代理的路由集合中移除。

**滚动更新。** 在运行时升级服务实现可以归约为受控的提供者迁移 [45, 46]。执行迁移时，先将新提供者作为额外的线程加载并向代理注册；新提供者进入 `ACTIVE` 后，再把流量逐步从旧提供者转移至新提供者（例如调整选择权重），待旧提供者不再承载任何在途请求后将其卸载。这种提供者迁移把传统上属于基础设施层的操作（例如容器编排和蓝绿部署）转化为应用层的组合模式。

**跨进程调用。** 服务代理也可以跨越进程边界应用 [47]。每个进程都承载自己的 Cordis 上下文与本地提供者；一个协调组件将这些进程连接起来，并把每个进程视为远程提供者。跨进程服务访问由保持接口不变的 RPC 机制介导，从而使分布对消费者透明。需要注意的是，跨进程调用会产生延迟，也可能在执行途中失败；若以同步方式公开这种调用，就会阻塞调用方。因此，准备跨进程公开的接口必须按照异步契约来设计。

## 5.2 访问控制与沙箱

对于由独立组件组装而成的应用，保障其安全需要两种互补机制：(1) 限制组件可以访问哪些依赖项；(2) 将不受信任的代码与宿主环境隔离。Cordis 通过依赖声明与拦截支持第一种机制；第二种机制则需要外部隔离边界。

**基于能力的访问控制。** 依赖访问机制（第 4.1.4 节）已经对经代理介导的属性构成了一种访问控制：组件只能访问已经声明的依赖项；访问未声明的依赖项会引发错误。这在结构上类似于基于能力的安全机制 [48, 49, 50]，其中的权限由持有引用而非环境权限所赋予。`inject` 声明充当能力请求，上下文代理则充当能力中介。由于这些请求是静态声明的，组件所需的全部代理介导能力在其运行前便已知，因此编排器可以在加载时审查并批准这些能力，而无须等到访问发生后才逐项发现。

这种介导可以借助拦截机制推广至细粒度策略。访问控制元数据可以由上下文携带，也可以由组件声明（定义 18）；调用依赖项时，提供者会查阅这些元数据，以决定是否允许请求。例如，文件系统依赖项可以携带元数据，声明某个组件可以读取或写入哪些路径，而提供者会根据这些元数据检查每次调用。由于这种拦截附着于上下文，而不驻留在任一参与方的代码中，编排器可以调整拦截，在不修改提供者的情况下约束任何组件对依赖项的访问；例如，只向社区组件授予数据库只读访问权限，同时让核心组件保留完整访问权限。此外，拦截只会影响依赖项的调用方式，而不会影响该依赖项是否得到满足，因此可以在运行时安装、重新配置或移除拦截，既不触发任何重新加载，也不扰动依赖图。



不受信任组件的沙箱化。当组件代码不可信时，语言层访问控制并不足够，因为恶意组件只要能够访问宿主运行时，就可以直接接触底层对象，从而使此类检查失去意义。真正的隔离需要一个超出语言层手段触及范围的执行边界，例如软件故障隔离 [51]、隔离的语言运行时、沙箱进程或虚拟化容器 [52]。无论采用何种机制，不受信任的组件都在自己的隔离上下文中运行，并通过桥接访问宿主提供的依赖项；这是第 5.1 节所述跨进程调用的推广：同样的透明性论证使这种桥接访问与本地注入不可区分。在宿主一侧，该桥接是一个普通线程，其能力范围可以通过上述访问控制加以收窄。

### 5.3 语言独立性与选择

尽管 Cordis 使用 TypeScript 实现，上下文范式本身却与语言无关：时空可组合性仅由两个可组合性维度定义，因此可以在任何沿这两个维度满足特定要求的语言中实现。下面依次分析每个维度的要求。

**时间可组合性。** 从最基本的层面看，时间可组合性需要闭包：可回退效应将一个操作与其逆操作配对，而逆操作及其所恢复的状态必须被捕获为值，以便在拆卸时重放。除此之外，还必须能够在运行时引入并撤回组件代码以及加载该代码所产生的副作用。

语言如何满足第二项要求，取决于其执行模型。在托管运行时中，这体现为可编程的模块注册表：已加载的模块可以从注册表中逐出，并在不再被引用后由垃圾回收器回收；例如，Node.js 就公开了这样的注册表<sup>6</sup>。原生代码不公开模块注册表，因此引入与撤回表现为显式的动态链接和取消链接（例如 Unix 上的 `dlopen/dlclose`，以及 Windows 上的 `LoadLibrary/FreeLibrary`）[53]，也就是把目标代码加载到运行中的进程里，随后再将其分离。WebAssembly 根据嵌入器的不同采取其中一条路径：在托管嵌入器（例如 JavaScript 宿主）下，模块实例由宿主的回收器回收；在原生嵌入器（例如 Wasmtime）下，则在嵌入器丢弃模块实例时将其释放。在所有这些机制中，可回退效应模型都把加载视为对上下文施加的效应，其逆操作会撤销模块所引入的符号、类型或处理程序注册。

**空间可组合性。** 空间可组合性要求具备一种机制，让组件能够声明自己的依赖项，并让运行时提供和注入这些依赖项。这可以归约为依赖注入（DI）问题 [39]；该问题在两个因语言而异的层次上显现：依赖项如何获得类型，以及对依赖项的访问如何得到介导。

在类型层面，语言应当为开发者提供一种表达良类型依赖访问的方式。消费者通过从上下文中读取键来取得余效应，所以，上下文类型（第 3.2.1 节）必须记录每个键的余效应。类型类（Haskell）[54] 和特征（Rust）[55] 允许提供者从自己的模块中通过实例（instance）或实现（impl）[56] 扩展上下文类型，从而满足这一要求。TypeScript 的模块扩充 [57] 同样允许提供者模块把声明合并到上下文类型中。

在运行时层面，依赖访问必须得到动态介导：随着提供者的加载与卸载，一个键背后的余效应可能发生变化，也可能在不同上下文中解析出不同结果。因此，语言需要一种以透明方式介入访问而不改动消费者代码的手段，例如 JavaScript 的 Proxy 对象 [58] 或 Python 的描述符协议（`__get__`）[59]。缺少这类原语时，运行时反射 [60, 61] 也能动态介导访问，代价则是类型安全与开发者体验。

在这两个层面上，元编程设施可以一并提供类型化与介导。注解 [62] 和装饰器把元数据附加到声明上，处理器再将其展开为负责介导访问的访问器；编译期元编程（例如 Rust 过程宏、Scala 宏 [63] 与 Zig 的 `comptime`）则会为每项依赖生成一份有类型的声明及相应的访问

<sup>6</sup>CommonJS 通过 `require.cache` 公开模块缓存；ES 模块没有公开的逐出 API，不过仍可通过引擎内部接口管理模块。

器，从而不再需要通用拦截原语。

#### 5.4 相互依赖与组件粒度

在反应式余效应模型中，依赖环只会让其中涉及的组件永久保持非活跃状态：给定两个组件  $A$  与  $B$ ，若  $A$  需要由  $B$  提供的键，而  $B$  又需要由  $A$  提供的键，那么二者的满足谓词都永远无法变为真。与并发系统中的死锁不同，这种状态可以静态预测（仅凭依赖声明即可），也不会产生运行时错误；它表现为静默的不激活，而非失败。

在实践中，多数看似相互依赖的关系都可以分解为粒度更细的组件，从而消除依赖环。考虑两个组件：一个服务器（提供网络接口）与一个访问控制器（实施授权策略）。两个组件双向交互：访问控制器介导到达服务器的请求，而服务器则公开用于修改访问控制策略的端点。整体式设计会让两个组件互相依赖。然而，这两个交互方向在逻辑上是彼此独立的关注点。将其分解后可得到四个组件：server-core、access-control-core、request-mediation（同时依赖两个核心组件，以便对传入请求实施访问控制），以及 policy-management（同时依赖两个核心组件，以便通过服务器公开策略修改功能）。通过这种方法可以消除依赖环，因为两个核心组件互不依赖；只有集成组件同时依赖二者。

原则上，这种分解总是可行，因为每一次双向交互都可以分解为彼此独立的单向绑定，但这会增加组件数量：一般而言，给定  $n$  个相互交互的组件，集成组件的数量可能随  $n$  呈平方增长，因为每一对相互交互的组件，都可能需要为每个交互方向分别设置一个组件。这不会影响正确性或运行时性能（组件是轻量级的），而且更细的粒度也可能带来益处：用户可以只加载所需的特定集成绑定，从而切实增强系统的可组合性。然而，它可能影响开发者体验：更多组件意味着更多配置、更多命名工作，以及理解依赖图时更高的认知开销。

缓解这种粒度成本是工程问题，而非理论问题。可行的策略包括捆绑（即把相关的细粒度组件组合为一个可安装单元）、基于约定的装配（即自动连接名称或类型符合某种模式的组件），以及脚手架工具（即根据声明式规格生成样板化的集成组件）。这些策略既保留了无环模型的形式保证，又把编写负担降低到更接近整体式设计的水平。

#### 5.5 依赖类型与版本管理

在形式模型中，依赖链接完全由键身份建立：提供键  $k$  的组件可以满足任何在依赖集中声明  $k$  的组件。类型族  $\mathcal{V}_k$  可以确保单个编译单元内部在类型层面保持一致，但在组件各自独立开发与构建时，这项保证便会失效；而这正是组件生态系统中的常见情形。保证的失效会引出两个不同的问题。

**接口漂移。** 提供者可能在不同版本间修改与  $k$  关联的接口，例如添加字段、变更方法签名或改变行为契约；与此同时，针对早期接口编译的消费者仍会声明同一个键  $k$ 。该依赖在余效应层面得到满足（ $k \in \text{dom}(\sigma)$ ），但运行时值已不再符合消费者的预期，从而导致类型错误、方法未找到故障或静默的行为偏离 [64]。

**键冲突。** 两个独立开发的提供者可能使用同一个键名  $k$  表示完全无关的接口。由于仅凭键身份便会建立链接，期待某个提供者接口的消费者会接受另一个提供者的值，而不进行任何兼容性检查。接口漂移中的提供者与消费者至少具有共同的演化谱系，键冲突则不同：预期类型与实际类型之间不存在任何关系，因而造成的故障不可预测，也难以诊断。

两个问题指向同一项缺口：余效应模型只提供名义链接（按键名链接），却不提供版本化链接或结构链接（按接口兼容性链接）[65]。下面讨论三种弥补这一缺口的方法，其顺序从与基础设施耦合最紧密的方法开始，直至最不依赖具体语言的方法。

**键命名空间化。** 把键空间从  $K$  扩展到  $K \times P$ , 其中  $P$  标识定义接口的包, 便可以从构造上消除键冲突: 独立开发但具有相同局部名称的接口会占据不同的键。这是最直接的解决方案, 但耦合也最紧密: 它把包命名空间嵌入形式模型本身, 使系统的键身份依赖于外部包注册表。

**对等依赖。** 耦合更松散的做法, 是通过宿主语言的包管理器声明版本约束 [66]。Cordis 目前采用的正是这种方法。组件依赖在语义上属于对等依赖: 组件不会在内部捆绑自己的依赖项, 而是期待运行时上下文提供这些依赖项。支持对等依赖的包管理器 (例如 npm) 可以强制执行版本兼容性: 如果提供某个键的包版本落在消费者声明的对等版本范围之外, 那么这种不兼容会在安装时被发现, 而不会演变为运行时故障。不过, 这种方法有两项局限: (1) 它依赖提供者忠实遵守语义化版本约定, 而这种约定无法强制执行; (2) 包管理器通常会把每项依赖解析为单一版本, 因而无法在一个应用中同时加载来自同一个包不同版本的组件。

**结构兼容性。** 一种完全与语言无关的方法, 是把成员关系检查  $k \in \text{dom}(\sigma)$  替换为兼容性谓词, 用来验证提供者的实际接口是否在结构上涵盖消费者的预期。这类似于结构子类型 [67]: 若所提供的接口是所需接口的子类型, 则提供者满足消费者。挑战在于如何以语言无关的方式定义这一谓词: 对记录类型而言, 结构兼容性很直接 (宽度子类型); 对行为契约而言则会变得复杂, 例如前置条件与后置条件 [68] 以及效应规格 [21]; 而一旦参数多态引入有界量化 [69], 这个问题就变得不可判定。

这三种方法处理了该问题的不同方面。如何设计一种统一的依赖模型, 在保持余效应模型所提供的动态组合保证的同时, 将这些方法结合起来, 仍是一个开放问题。

## 6 相关工作

动态可组合性与若干已有研究领域相交。本节综述其中最相关的研究脉络, 并逐一阐明本文贡献与它们的区别。

### 6.1 效应与余效应系统

第 2 节回顾了作为本文工作理论支柱的效应与余效应。三条研究路线从与 Cordis 相关的方向扩展了二者: 将效应重新阐释为上下文必须提供的能力; 赋予效应可逆语义而非解释语义; 以及在单一分级规约下统一效应与余效应。

**作为能力的代数效应。** 代数效应 (第 2.1 节) 使效应操作对类型系统可见。与本文工作最近的扩展是 Brachthäuser 等人的 Effekt 语言, 它将效应类型重新阐释为能力 [70, 71]: 效应类型表达的是计算要求上下文提供什么, 而不是计算可能产生什么副作用。同一种重新阐释也已进入主流实践: Effect-TS 库 [72] 在专门的类型参数中记录效应的上下文需求, 使计算的类型能够说明其外围上下文必须提供哪些服务; 与 Effekt 一样, 它通过静态类型上的解释来消解效应, 而不是将效应回退。这一视角与本文一样, 将上下文视为能力的中介。Cordis 与 Effekt 在两个方面存在差异。就目的而言, 代数效应令效应可见, 是为了实现模块化解释, 从而为同一个操作赋予多种处理器语义; Cordis 令效应可见, 则是为了实现跟踪与回退, 即为每一次上下文变换配以逆变换。就应用环境而言, Effekt 在类型层面静态规约效应, 默认采用基于作用域的推理, 其中能力是二等的, 并受限其词法作用域; 它又通过装箱恢复能力的一等用法, 以类型跟踪捕获的能力, 从而解除这一限制。Cordis 则在运行时规约效应, 目标是在组件移除时完整恢复资源。



**可逆效应语义。** 另一条研究路线赋予效应可逆语义，而非解释语义。Heunen 等人 [73] 改造 Hughes 的箭头，以 dagger 箭头和逆箭头在可逆环境中建模副作用，涵盖序列化和可变存储等操作存在逆的效应。这是与本文可回退效应最接近的形式化论述：二者都为每个效应配以逆，而不是通过处理器消解效应。区别在于可逆性位于何处。Heunen 等人采用指称式的范畴论框架，其中可逆性是一项全局性质；由于每项计算都可逆，可逆性由构造保证，而逆则从范畴结构中恢复。Cordis 则在运行时跟踪逆：它并不要求整个计算可逆，只要求每个原子效应存在逆，并由组合推导任意复合效应的逆（第 3.1 节）。

**统一效应与余效应的分级类型。** Orchard 等人 [74] 提出分级模态类型，以此作为同时涵盖效应推理（通过分级单子）和余效应推理（通过分级余单子）的统摄概念，并在 Granule 语言中予以实现，表明单一类型系统可以同时跟踪计算做什么以及计算需要什么。较新的工作进一步将余效应扩展到命令式的 Java 风格语言 [75, 76] 以及 call-by-push-value [77]。这些工作全都在类型层面运作：效应与余效应是编译时检查的静态注解，其作用域由词法结构固定。本文的贡献与此类分析正交：本文将同样的两个概念提升为运行时机制，使 Cordis 能够处理动态组合。随着已加载组件的集合不断演变，时间回退与空间依赖会被重新求解，而不是针对固定的程序文本一次性确定。

## 6.2 编程范式

第 3.4.2 节确立了上下文范式，即通过显式上下文中介效应与余效应的规约。这里有必要明确比较两种既有范式：一种与本文共享术语，另一种与本文共享对横切关注点的处理方式。

**面向上下文编程。** 面向上下文编程（context-oriented programming, COP）[78, 79] 为语言配备层；层由方法和类的部分定义构成，并根据执行上下文在运行时激活或停用，使行为能够自适应，而无须由基础代码点名其上下文依赖 [80]。COP 与 Cordis 都把上下文视为一等、可在运行时改变的实体，也都动态激活和停用行为，但二者的相似仅停留在名称上。在 COP 中，“上下文”表示周围的执行情境（例如位置、用户或模式），激活操作会改变动态作用域范围内的方法派发；层既不跟踪自身引发的副作用，也不回退这些副作用，而且激活并不由依赖满足性决定。在 Cordis 中，上下文是中介效应与余效应的  $\Gamma_\infty$  实体：激活过程运行组件的可回退效应，并由反应式余效应的满足性驱动（第 3.2 节）；停用则将这些效应完整回退。COP 改变运行哪一种行为；Cordis 则组合并回退组件安装的效应与依赖。二者的差异体现为一种取舍。COP 将激活融入宿主语言的方法派发，以语言特定性为代价获得动态作用域的层范围；Cordis 作为与语言无关的叠 layers，则在共享上下文上反应式地求解激活。因此，Cordis 只能将 COP 中全局、值驱动的部分表达为余效应，也就是依上下文在多个实现之间选择，而不能表达动态作用域激活。

**面向切面编程。** 面向切面编程（aspect-oriented programming, AOP）[81, 82] 把横切关注点模块化为切面：切点对基础程序中选出的连接点进行量化，并在每个连接点织入通知。Cordis 处理的是同一个问题，即若不加处理便会散落在各组件中的上下文相关行为；但在 Cordis 中，与切面对应的是余效应，也就是多个组件声明共同依赖的共享中介点，因而可以在该处重塑横切行为，而无须编辑任何组件。这两种范式继而在两个维度上有所不同。第一，**声明与无感知**：AOP 切点具有无感知性并进行量化，可匹配任意连接点，而这些连接点处的代码并不知道自身受到通知；Cordis 则把横切限制在每个组件主动声明的余效应上，因此其作用范围恰好就是这一声明面。这带来了确定性与可追踪性：应用编排器可以在配置层检查并治理哪些内容会横切组件，而无须阅读或分析组件源码；AOP 关注点则只有通过对其进行量化的切面才可辨识。第二，**生命周期集成**：Cordis 中的横切变化由组件的效应承载，在组件卸载时回退，

并以反应式方式传播给依赖该组件的其他组件，因此它是动态组合模型内部的一次操作；动态 AOP 系统 [83, 84] 同样可以在运行时织入和解除织入，但这是独立操作，既不绑定于组件生命周期，也不会触发被通知代码之间的重新求解。

### 6.3 时间可组合性

时间可组合性关注如何在运行中的程序内替换或移除组件，同时恢复它所安装的效应。既有方法可以按如何处理离场组件的状态与效应来划分：把状态向前迁移到后继版本；通过开发者编写的清理逻辑恢复效应；或者在预先固定的作用域内自动反转效应。

**有状态的前向迁移。** 一大类系统在运行中的程序里替换组件时，会跨版本向前迁移组件状态，以避免停机。这些系统全都遵守同一项时序规约：只有当组件到达安全、无交互的时间点后，才能将其换出。Kramer 和 Magee 将这项判据确立为**静止性** (quiescence) [45]，Vandewoude 等人随后将其放宽为干扰更小的**安宁性** (tranquility) [46]；本文的滚动更新模式（第 5.1 节）通过在卸载提供者前排空在途请求来落实这一判据。动态软件更新 (dynamic software updating, DSU) 继而通过手写转换函数向前迁移状态：Hicks 等人面向 C 的通用 DSU [85]、Stoyle 等人通过 con-freeness 分析确定类型安全的更新点 [86]，以及 Hayden 等人的 Kitsune [87]，全都会将旧版本数据映射到新版本表示，在原地继承堆对象、打开的文件和连接，同时重新初始化未被迁移的部分。同一规约也延伸至持久状态：Overeem 等人 [88] 使用手写升级操作，在保持系统可用的同时，转换运行中事件存储在不同模式版本之间的数据。Erlang/OTP [14] 在进程层面采取同样的立场，通过 `code_change/3` 迁移状态，并通过重启受监督进程从故障中恢复，而不是回退进程的效应；JavaScript 的热模块替换（例如 webpack [42] 和 Vite [43]）在模块层面采取同样的做法，跨重载通过 `module.hot` 或 `import.meta.hot` API 将状态向前传递。与 Cordis 的模块替换（第 4.2 节）相比，这些方法对内存状态的迁移更为平滑：Cordis 回退旧组件受到跟踪的效应，并从干净状态重新应用新组件的效应；因此，除非组件自身的内存状态被置于生命周期更长的依赖项中，否则这些状态无法在重载后继续存在。在可回退效应之上叠加 DSU 式前向迁移仍是未来工作。不过，Cordis 的方法在两个方面更为通用：它不需要 DSU 和热模块替换所需的手写迁移函数；它还支持彻底卸载组件并恢复其资源，而不只是原地更新组件。

**开发者编写的恢复逻辑。** 第二类方法通过开发者手写的清理或补偿逻辑来恢复组件效应。插件生命周期约定（例如 OSGi [44]、Eclipse 的扩展点、IntelliJ 和 VSCode）把清理工作交给开发者编写的卸载回调；命令模式 [89] 把操作与用于撤销/重做栈的 `undo` 方法封装在一起；Saga 模型 [37] 把长事务组织为一系列步骤，并为每一步配以补偿操作；代数效应处理器可以附加在拆卸时运行的终结器 [90]；事件溯源 [91] 则通过追加补偿事件撤销状态，而不执行真正的逆。在所有这些方法中，提供逆都是一项不受强制约束且与操作相分离的职责，因此，遗漏逆会悄无声息地泄漏资源（第 1.2.1 节记录了实证情况）。React 的 `useEffect` 钩子 [92] 最接近在结构上为效应配以逆：它返回一个清理函数，运行时会在每次重新执行之前以及卸载时调用该函数。其不足之处在于可组合性：钩子只能在组件或另一个钩子的顶层调用，绝不能在条件分支、循环或嵌套函数内部调用；其效应体既不能采用异步函数，也不能采用迭代器。因此，效应无法由其他效应组装而成，也无法与控制流交错，因而没有可供推导复合效应之逆的结构。Cordis 效应没有此类限制：它们是可以自由组合、可以异步运行的普通操作；只有每个原子效应需要手写一个逆，任意复合效应的逆都由组合推导而来。因此，组装已有的效应根本无须编写新的逆。每个效应与其逆的这种结构性配对，使完整恢复成为系统不变量，而不再仰赖开发者自律。



静态作用域内的反转。第三类方法通过构造自动反转效应，但把反转限制在预先固定的作用域内。软件事务内存 [93, 94] 源自硬件事务内存 [95]，它记录读写日志，使一组内存操作要么提交，要么中止并把内存回滚到事务前的状态。从 Landauer 和 Bennett 的热力学分析 [96, 97] 到 Janus 等可逆语言 [98]，可逆计算更进一步，使完整计算中的每一步在全局上都可逆。线性类型 [99]、RAII[4] 以及 Rust 的所有权系统 [55] 把资源释放绑定到词法区域。这些方法都静态固定了反转的作用域与影响范围；相比之下，Cordis 不会预先固定这类作用域：它在组件生命周期内回退任意上下文操作，并将词法资源管理视为互补机制，适合管理单个组件内部的局部资源。

## 6.4 空间可组合性

空间可组合性关注如何声明并绑定一个组件对其他组件的依赖。既有机制可以按绑定如何响应变化来划分：只在初始化时连接依赖；响应整个组件的可用性；或者以单个值为粒度传播变化。

初始化时的依赖连接。两类已有机制会在初始化时连接组件。依赖注入框架 [39]（例如 Spring[100]、Guice、Angular 和 Inversify）在初始化时向组件注入依赖；UI 框架的上下文机制（例如 Vue.js 的 `provide/inject` 和 React Context API）则沿组件树传递依赖。其中一些机制支持动态作用域（例如 Spring 的原型/请求作用域和 Angular 的层次化注入器），但两者都不会作出反应式的重新求解：当提供者在运行时被替换或撤回，已有的依赖方既不会停用，也不会重新初始化；它们也都不提供本文组件状态机所具有的生命周期管理。Cordis 的反应式余效应（第 3.2 节）弥补了这一缺失：只要满足性谓词发生变化，通知机制就会触发生命周期转换。

响应可用性的组件模型。与本文反应式余效应最接近的先例会对服务可用性作出反应。OSGi 的 Declarative Services 和 iPOJO[101, 102] 允许组件声明所提供和所需要的服务；当服务出现或消失时，运行时会自动激活或停用组件。iPOJO 的 Gravity 项目 [102] 明确以根据服务可用性的变化自主进行运行时适配为目标，其 `provide/require` 模型直接预示了 Cordis 的 `ctx.provide/ctx.get` 模式。R-OSGi[47] 通过 RPC 将同一种抽象透明地扩展到分布式环境，把网络故障映射为服务撤回事件；第 5.1 节将这种模式作为 Cordis 模型的一项扩展加以讨论。所有这些系统都通过停用回调进行恢复，而这种回调有两点局限。第一，回调需要手写，因而资源安全依赖于开发者规约；遗漏回调会悄无声息地造成泄漏。第二，回调是同步的；如果拆卸过程需要与即将离场的依赖项进行异步交互，框架却没有协议可供等待交互完成，只能针对可能已过时的引用进行阻塞等待。Cordis 的反应式余效应弥补了这两处缺陷：停用操作会回退依赖方累积的效应；其惯性 UNLOAD 状态（第 3.3.4 节）会让异步拆卸完整运行，然后才响应后续变化。

值级反应性。函数式反应式编程（functional reactive programming, FRP）[103] 及其现代实现（例如 SolidJS 中的信号 [104, 105]、Vue 的反应性系统和 Angular Signals）以值级粒度传播变化：当信号发生变化时，派生计算会同步地或在调度器下重新求值 [106]。Cordis 的反应式余效应则以组件级粒度运作，加入了值级传播所不建模的异步生命周期语义。二者互为补充，而非相互竞争：Cordis 余效应本身可以承载反应式值，组件只根据自身实际消费的部分进行更新，由此把组件级反应性细化为跨越两个层级、粒度更细的反应式余效应。

## 7 结论

本文把效应与余效应这两个类型论概念提升为运行时机制，为动态可组合性奠定了形式化基础。可回退效应通过为上下文变换配备显式的逆来解决时间可组合性，并证明效应跟踪保持组合运算，从而保证组件移除时完整恢复状态。反应式余效应通过形式化有类型的依赖上下文来解决空间可组合性；这类上下文具备基于满足性的通知、余效应隔离和余效应拦截。组件生命周期模型通过适配多种控制流的正交扩展，使两种机制之间的交互具备操作语义。它们共同导出统一的上下文类型，把效应上下文与余效应上下文整合成连贯的编程范式。

本文将这一范式实现为 Cordis 元框架：其核心库提供效应跟踪和余效应求解，声明式组件加载器则提供配置协调与热模块替换。Koishi 案例研究在拥有 4000 多个社区插件的生产系统中验证了 Cordis 的设计。

除由人类维护的插件生态外，自演化智能体框架（第 1.2.2 节）也是很有吸引力的未来验证方向。在这种环境中，AI 智能体几乎不受人类监督，持续生成和替换自己的框架组件。把 Cordis 应用于此类环境，将验证在快速替换组件时实现完整恢复的时间保证，也将验证在拓扑频繁变化时实现依赖协调的空间保证。此类验证将表明，该范式可以作为智能体框架及其他自主系统实现可恢复、协调且持续自演化的基础。

## 参考文献

- [1] D. L. Parnas, “On the criteria to be used in decomposing systems into modules,” *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972, doi: [10.1145/361598.361623](https://doi.org/10.1145/361598.361623).
- [2] D. Birsan, “On Plug-ins and Extensible Architectures,” *ACM Queue*, vol. 3, no. 2, pp. 40–46, 2005, doi: [10.1145/1053331.1053345](https://doi.org/10.1145/1053331.1053345).
- [3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016, doi: [10.1145/2890784](https://doi.org/10.1145/2890784).
- [4] B. Stroustrup, *The Design and Evolution of C++*. Addison-Wesley, 1994.
- [5] S. Marlow, S. Peyton Jones, A. Moran, and J. Reppy, “Asynchronous Exceptions in Haskell,” in *Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation*, in PLDI '01. New York, NY, USA: Association for Computing Machinery, 2001, pp. 274–285. doi: [10.1145/378795.378858](https://doi.org/10.1145/378795.378858).
- [6] C. Szyperski, *Component Software: Beyond Object-Oriented Programming*, 2nd ed. Addison-Wesley, 2002.
- [7] R. Lopopolo, “Harness Engineering: Leveraging Codex in an Agent-First World.” [Online]. Available: <https://openai.com/index/harness-engineering/>
- [8] Anthropic, “Harness Design for Long-Running Application Development.” [Online]. Available: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- [9] L. Wang *et al.*, “A Survey on Large Language Model Based Autonomous Agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024, doi: [10.1007/s11704-024-40231-1](https://doi.org/10.1007/s11704-024-40231-1).
- [10] Y. Qin *et al.*, “Tool Learning with Foundation Models,” *ACM Computing Surveys*, 2025, doi: [10.1145/3704435](https://doi.org/10.1145/3704435).

- [11] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” *CoRR*, 2023.
- [12] T. Guo *et al.*, “Large Language Model Based Multi-Agents: A Survey of Progress and Challenges,” in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, in IJCAI 2024. 2024, pp. 8048–8057. doi: [10.24963/ijcai.2024/890](https://doi.org/10.24963/ijcai.2024/890).
- [13] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, “Large Language Models as Tool Makers,” in *Proceedings of the Twelfth International Conference on Learning Representations*, in ICLR 2024. 2024.
- [14] J. Armstrong, “Making Reliable Distributed Systems in the Presence of Software Errors,” Doctoral dissertation, 2003.
- [15] E. Moggi, “Notions of computation and monads,” *Information and Computation*, vol. 93, no. 1, pp. 55–92, 1991, doi: [10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4).
- [16] G. Plotkin and J. Power, “Adequacy for Algebraic Effects,” in *Foundations of Software Science and Computation Structures*, F. Honsell and M. Miculan, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 1–24.
- [17] T. Petricek, D. Orchard, and A. Mycroft, “Coeffects: unified static analysis of context-dependence,” in *Proceedings of the 40th International Conference on Automata, Languages, and Programming - Volume Part II*, in ICALP’13. Riga, Latvia: Springer-Verlag, 2013, pp. 385–397. doi: [10.1007/978-3-642-39212-2\\_35](https://doi.org/10.1007/978-3-642-39212-2_35).
- [18] M. Gaboardi, S.-y. Katsumata, D. Orchard, F. Breuvar, and T. Uustalu, “Combining effects and coeffects via grading,” in *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming*, in ICFP 2016. Nara, Japan: Association for Computing Machinery, 2016, pp. 476–489. doi: [10.1145/2951913.2951939](https://doi.org/10.1145/2951913.2951939).
- [19] A. Church, “A Formulation of the Simple Theory of Types,” *The Journal of Symbolic Logic*, vol. 5, no. 2, pp. 56–68, 1940, doi: [10.2307/2266170](https://doi.org/10.2307/2266170).
- [20] B. C. Pierce, *Types and Programming Languages*. MIT Press, 2002.
- [21] J. M. Lucassen and D. K. Gifford, “Polymorphic Effect Systems,” in *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL ’88. San Diego, California, USA: Association for Computing Machinery, 1988, pp. 47–57. doi: [10.1145/73560.73564](https://doi.org/10.1145/73560.73564).
- [22] P. Wadler, “Monads for functional programming,” in *Program Design Calculi*, M. Broy, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 233–264.
- [23] G. Plotkin and J. Power, “Notions of Computation Determine Monads,” in *Foundations of Software Science and Computation Structures*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 342–356. doi: [10.1007/3-540-45931-6\\_24](https://doi.org/10.1007/3-540-45931-6_24).
- [24] G. Plotkin and M. Pretnar, “Handlers of Algebraic Effects,” in *Programming Languages and Systems (ESOP)*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 80–94. doi: [10.1007/978-3-642-00590-9\\_7](https://doi.org/10.1007/978-3-642-00590-9_7).

- [25] M. Pretnar, “An Introduction to Algebraic Effects and Handlers. Invited tutorial paper,” *Electron. Notes Theor. Comput. Sci.*, vol. 319, no. C, pp. 19–35, Dec. 2015, doi: [10.1016/j.entcs.2015.12.003](https://doi.org/10.1016/j.entcs.2015.12.003).
- [26] D. Leijen, “Koka: Programming with Row Polymorphic Effect Types,” *Electronic Proceedings in Theoretical Computer Science*, vol. 153, pp. 100–126, Jun. 2014, doi: [10.4204/eptcs.153.8](https://doi.org/10.4204/eptcs.153.8).
- [27] D. Leijen, “Type directed compilation of row-typed algebraic effects,” in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, in POPL ’17. Paris, France: Association for Computing Machinery, 2017, pp. 486–499. doi: [10.1145/3009837.3009872](https://doi.org/10.1145/3009837.3009872).
- [28] A. Bauer and M. Pretnar, “Programming with algebraic effects and handlers,” *Journal of Logical and Algebraic Methods in Programming*, vol. 84, no. 1, pp. 108–123, Jan. 2015, doi: [10.1016/j.jlamp.2014.02.001](https://doi.org/10.1016/j.jlamp.2014.02.001).
- [29] K. Sivaramakrishnan *et al.*, “Retrofitting parallelism onto OCaml,” *Proc. ACM Program. Lang.*, vol. 4, no. ICFP, Aug. 2020, doi: [10.1145/3408995](https://doi.org/10.1145/3408995).
- [30] T. Petricek, D. Orchard, and A. Mycroft, “Coeffects: a calculus of context-dependent computation,” in *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, in ICFP ’14. Gothenburg, Sweden: Association for Computing Machinery, 2014, pp. 123–135. doi: [10.1145/2628136.2628160](https://doi.org/10.1145/2628136.2628160).
- [31] T. Uustalu and V. Vene, “Comonadic Notions of Computation,” *Electronic Notes in Theoretical Computer Science*, vol. 203, no. 5, pp. 263–284, 2008, doi: [10.1016/j.entcs.2008.05.029](https://doi.org/10.1016/j.entcs.2008.05.029).
- [32] A. Brunel, M. Gaboardi, D. Mazza, and S. Zdanczewicz, “A Core Quantitative Coeffect Calculus,” in *Proceedings of the 23rd European Symposium on Programming Languages and Systems - Volume 8410*, Berlin, Heidelberg: Springer-Verlag, 2014, pp. 351–370. doi: [10.1007/978-3-642-54833-8\\_19](https://doi.org/10.1007/978-3-642-54833-8_19).
- [33] J. Reed and B. C. Pierce, “Distance makes the types grow stronger: a calculus for differential privacy,” *SIGPLAN Not.*, vol. 45, no. 9, pp. 157–168, Sep. 2010, doi: [10.1145/1932681.1863568](https://doi.org/10.1145/1932681.1863568).
- [34] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A core calculus of dependency,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL ’99. San Antonio, Texas, USA: Association for Computing Machinery, 1999, pp. 147–160. doi: [10.1145/292540.292555](https://doi.org/10.1145/292540.292555).
- [35] D. E. Denning, “A lattice model of secure information flow,” *Commun. ACM*, vol. 19, no. 5, pp. 236–243, May 1976, doi: [10.1145/360051.360056](https://doi.org/10.1145/360051.360056).
- [36] U. Dal Lago and F. Gavazzo, “A relational theory of effects and coeffects,” *Proc. ACM Program. Lang.*, vol. 6, no. POPL, Jan. 2022, doi: [10.1145/3498692](https://doi.org/10.1145/3498692).
- [37] H. Garcia-Molina and K. Salem, “Sagas,” in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, in SIGMOD ’87. 1987, pp. 249–259. doi: [10.1145/38713.38742](https://doi.org/10.1145/38713.38742).

- [38] R. Johnson and B. Foote, “Designing Reusable Classes,” *Journal of Object-Oriented Programming*, vol. 1, pp. 22–35, 1988.
- [39] M. Fowler, “Inversion of Control Containers and the Dependency Injection pattern.” [Online]. Available: <https://martinfowler.com/articles/injection.html>
- [40] A. M. Pitts and I. D. B. Stark, “Observable Properties of Higher Order Functions that Dynamically Create Local Names, or What’s New?,” in *Mathematical Foundations of Computer Science 1993 (MFCS 1993)*, in Lecture Notes in Computer Science, vol. 711. Springer, 1993, pp. 122–141. doi: [10.1007/3-540-57182-5\\_8](https://doi.org/10.1007/3-540-57182-5_8).
- [41] R. P. James and A. Sabry, “Yield: Mainstream Delimited Continuations,” in *First International Workshop on the Theory and Practice of Delimited Continuations (TPDC 2011)*, 2011, pp. 20–32.
- [42] webpack, “Hot Module Replacement.” [Online]. Available: <https://webpack.js.org/api/hot-module-replacement/>
- [43] Vite, “HMR API.” [Online]. Available: <https://vite.dev/guide/api-hmr>
- [44] OSGi Alliance, *OSGi Core Release 8*. OSGi Alliance, 2020.
- [45] J. Kramer and J. Magee, “The Evolving Philosophers Problem: Dynamic Change Management,” *IEEE Transactions on Software Engineering*, vol. 16, no. 11, pp. 1293–1306, 1990, doi: [10.1109/32.60317](https://doi.org/10.1109/32.60317).
- [46] Y. Vandewoude, P. Ebraert, Y. Berbers, and T. D’Hondt, “Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates,” *IEEE Transactions on Software Engineering*, vol. 33, no. 12, pp. 856–868, 2007, doi: [10.1109/tse.2007.70733](https://doi.org/10.1109/tse.2007.70733).
- [47] J. S. Rellermeyer, G. Alonso, and T. Roscoe, “R-OSGi: Distributed Applications Through Software Modularization,” in *Proceedings of the ACM/IFIP/USENIX 8th International Middleware Conference*, in Middleware ’07. 2007, pp. 1–20. doi: [10.1007/978-3-540-76778-7\\_1](https://doi.org/10.1007/978-3-540-76778-7_1).
- [48] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multiprogrammed Computations,” *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.
- [49] M. S. Miller, K.-P. Yee, and J. Shapiro, “Capability Myths Demolished,” in *Technical Report SRL2003-02*, 2003.
- [50] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, “Capsicum: Practical Capabilities for UNIX,” in *Proceedings of the 19th USENIX Security Symposium*, 2010, pp. 29–46.
- [51] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, “Efficient Software-Based Fault Isolation,” in *Proceedings of the 14th ACM Symposium on Operating Systems Principles*, in SOSP ’93. 1993, pp. 203–216. doi: [10.1145/168619.168635](https://doi.org/10.1145/168619.168635).
- [52] A. Barth, A. P. Felt, P. Saxena, and A. Boodman, “Protecting Browsers from Extension Vulnerabilities,” in *Proceedings of the 17th Annual Network and Distributed System Security Symposium*, in NDSS ’10. 2010.



- [53] W. W. Ho and R. A. Olsson, “An Approach to Genuine Dynamic Linking,” *Software: Practice and Experience*, vol. 21, no. 4, pp. 375–390, 1991, doi: [10.1002/SPE.4380210404](https://doi.org/10.1002/SPE.4380210404).
- [54] P. Wadler and S. Blott, “How to Make Ad-hoc Polymorphism Less Ad Hoc,” in *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL ’89. 1989, pp. 60–76. doi: [10.1145/75277.75283](https://doi.org/10.1145/75277.75283).
- [55] N. D. Matsakis and F. S. K. II, “The Rust Language and Type System,” in *ACM SIGPLAN ML Family Workshop*, Gothenburg, Sweden, Sep. 2014.
- [56] D. Dreyer, R. Harper, M. M. T. Chakravarty, and G. Keller, “Modular Type Classes,” in *Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL ’07. 2007, pp. 63–70. doi: [10.1145/1190216.1190229](https://doi.org/10.1145/1190216.1190229).
- [57] Microsoft, “Declaration Merging.” [Online]. Available: <https://www.typescriptlang.org/docs/handbook/declaration-merging.html>
- [58] T. Van Cutsem and M. S. Miller, “Proxies: Design Principles for Robust Object-oriented Intercession APIs,” in *Proceedings of the 6th Symposium on Dynamic Languages*, in DLS ’10. 2010, pp. 59–72. doi: [10.1145/1869631.1869638](https://doi.org/10.1145/1869631.1869638).
- [59] R. Hettinger, “Descriptor HowTo Guide.” [Online]. Available: <https://docs.python.org/3/howto/descriptor.html>
- [60] P. Maes, “Concepts and Experiments in Computational Reflection,” in *Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, 1987, pp. 147–155. doi: [10.1145/38765.38821](https://doi.org/10.1145/38765.38821).
- [61] G. Bracha and D. M. Ungar, “Mirrors: design principles for meta-level facilities of object-oriented programming languages,” in *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2004, pp. 331–344. doi: [10.1145/1028976.1029004](https://doi.org/10.1145/1028976.1029004).
- [62] R. Rouvoy and P. Merle, “Leveraging component-based software engineering with Fraclet,” *Annals of Telecommunications*, vol. 64, no. 1–2, pp. 65–79, 2009, doi: [10.1007/s12243-008-0072-z](https://doi.org/10.1007/s12243-008-0072-z).
- [63] E. Burmako, “Scala Macros: Let Our Powers Combine!,” in *Proceedings of the 4th Workshop on Scala*, in SCALA@ECOOP ’13. 2013, pp. 3:1–3:10. doi: [10.1145/2489837.2489840](https://doi.org/10.1145/2489837.2489840).
- [64] S. Raemaekers, A. van Deursen, and J. Visser, “Semantic Versioning and Impact of Breaking Changes in the Maven Repository,” *Journal of Systems and Software*, vol. 129, pp. 140–158, 2017, doi: [10.1016/j.jss.2016.04.008](https://doi.org/10.1016/j.jss.2016.04.008).
- [65] P. Lam, J. Dietrich, and D. J. Pearce, “Putting the Semantics into Semantic Versioning,” in *Proceedings of the 2020 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, in Onward! ’20. 2020, pp. 157–179. doi: [10.1145/3426428.3426922](https://doi.org/10.1145/3426428.3426922).

- [66] P. Abate, R. Di Cosmo, R. Treinen, and S. Zacchiroli, “Dependency Solving: A Separate Concern in Component Evolution Management,” *Journal of Systems and Software*, vol. 85, no. 10, pp. 2228–2240, 2012, doi: [10.1016/j.jss.2012.02.018](https://doi.org/10.1016/j.jss.2012.02.018).
- [67] L. Cardelli, “Structural Subtyping and the Notion of Power Type,” in *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL ’88, 1988, pp. 70–79. doi: [10.1145/73560.73566](https://doi.org/10.1145/73560.73566).
- [68] B. Meyer, “Applying “Design by Contract”,” *Computer*, vol. 25, no. 10, pp. 40–51, 1992, doi: [10.1109/2.161279](https://doi.org/10.1109/2.161279).
- [69] B. C. Pierce, “Bounded Quantification is Undecidable,” *Information and Computation*, vol. 112, no. 1, pp. 131–165, 1994, doi: [10.1006/inco.1994.1055](https://doi.org/10.1006/inco.1994.1055).
- [70] J. I. Brachthäuser, P. Schuster, and K. Ostermann, “Effects as capabilities: effect handlers and lightweight effect polymorphism,” *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, 2020, doi: [10.1145/3428194](https://doi.org/10.1145/3428194).
- [71] J. I. Brachthäuser, P. Schuster, E. Lee, and A. Boruch-Gruszecki, “Effects, capabilities, and boxes: from scope-based reasoning to type-based reasoning and back,” *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA1, 2022, doi: [10.1145/3527320](https://doi.org/10.1145/3527320).
- [72] Effect, “Effect: A TypeScript library for building robust applications.” [Online]. Available: <http://effect.website/>
- [73] C. Heunen, R. Kaarsgaard, and M. Karvonen, “Reversible Effects as Inverse Arrows,” in *Proceedings of the Thirty-Fourth Conference on the Mathematical Foundations of Programming Semantics (MFPS XXXIV)*, in Electronic Notes in Theoretical Computer Science, vol. 341, 2018, pp. 179–199. doi: [10.1016/j.entcs.2018.11.009](https://doi.org/10.1016/j.entcs.2018.11.009).
- [74] D. Orchard, V.-B. Liepelt, and H. Eades III, “Quantitative program reasoning with graded modal types,” *Proc. ACM Program. Lang.*, vol. 3, no. ICFP, 2019, doi: [10.1145/3341714](https://doi.org/10.1145/3341714).
- [75] R. Bianchini, F. Dagnino, P. Giannini, E. Zucca, and M. Servetto, “Coeffects for sharing and mutation,” *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA2, Oct. 2022, doi: [10.1145/3563319](https://doi.org/10.1145/3563319).
- [76] R. Bianchini, F. Dagnino, P. Giannini, and E. Zucca, “A Java-like calculus with heterogeneous coeffects,” *Theoretical Computer Science*, vol. 971, p. 114063, 2023, doi: <https://doi.org/10.1016/j.tcs.2023.114063>.
- [77] C. Torczon, E. Suárez Acevedo, S. Agrawal, J. Velez-Ginorio, and S. Weirich, “Effects and Coeffects in Call-by-Push-Value,” *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, Oct. 2024, doi: [10.1145/3689750](https://doi.org/10.1145/3689750).
- [78] R. Hirschfeld, P. Costanza, and O. Nierstrasz, “Context-oriented Programming,” *Journal of Object Technology*, vol. 7, no. 3, pp. 125–151, 2008, doi: [10.5381/jot.2008.7.3.a4](https://doi.org/10.5381/jot.2008.7.3.a4).
- [79] P. Costanza and R. Hirschfeld, “Language constructs for context-oriented programming: an overview of ContextL,” in *Proceedings of the 2005 Symposium on Dynamic Languages (DLS ’05)*, ACM, 2005, pp. 1–10. doi: [10.1145/1146841.1146842](https://doi.org/10.1145/1146841.1146842).

- [80] G. Salvaneschi, C. Ghezzi, and M. Pradella, “Context-oriented programming: A software engineering perspective,” *Journal of Systems and Software*, vol. 85, no. 8, pp. 1801–1817, 2012, doi: [10.1016/j.jss.2012.03.024](https://doi.org/10.1016/j.jss.2012.03.024).
- [81] G. Kiczales *et al.*, “Aspect-Oriented Programming,” in *ECOOP’97 — Object-Oriented Programming, 11th European Conference*, in Lecture Notes in Computer Science, vol. 1241. Springer, 1997, pp. 220–242. doi: [10.1007/BFb0053381](https://doi.org/10.1007/BFb0053381).
- [82] G. Kiczales, E. Hilsdale, J. Hugunin, M. Kersten, J. Palm, and W. G. Griswold, “An Overview of AspectJ,” in *ECOOP 2001 — Object-Oriented Programming, 15th European Conference*, in Lecture Notes in Computer Science, vol. 2072. Springer, 2001, pp. 327–353. doi: [10.1007/3-540-45337-7\\_18](https://doi.org/10.1007/3-540-45337-7_18).
- [83] A. Popovici, T. Gross, and G. Alonso, “Dynamic Weaving for Aspect-Oriented Programming,” in *Proceedings of the 1st International Conference on Aspect-Oriented Software Development (AOSD 2002)*, ACM, 2002, pp. 141–147. doi: [10.1145/508386.508404](https://doi.org/10.1145/508386.508404).
- [84] J. Bonér, “What Are the Key Issues for Commercial AOP Use: How Does AspectWerkz Address Them?,” in *Proceedings of the 3rd International Conference on Aspect-Oriented Software Development (AOSD 2004)*, ACM, 2004, pp. 5–6. doi: [10.1145/976270.976273](https://doi.org/10.1145/976270.976273).
- [85] M. Hicks, J. T. Moore, and S. Nettles, “Dynamic Software Updating,” in *Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation*, in PLDI ’01. 2001, pp. 13–23. doi: [10.1145/378795.378798](https://doi.org/10.1145/378795.378798).
- [86] G. Stoyle, M. Hicks, G. Bierman, P. Sewell, and I. Neamtiu, “Mutatis Mutandis: Safe and Predictable Dynamic Software Updating,” in *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL ’05. 2005, pp. 183–194. doi: [10.1145/1040305.1040321](https://doi.org/10.1145/1040305.1040321).
- [87] C. M. Hayden, K. Saur, E. K. Smith, and M. Hicks, “Kitsune: Efficient, General-Purpose Dynamic Software Updating for C,” *ACM Trans. Program. Lang. Syst.*, vol. 36, no. 4, 2014, doi: [10.1145/2629460](https://doi.org/10.1145/2629460).
- [88] M. Overeem, M. Spoor, and S. Jansen, “The Dark Side of Event Sourcing: Managing Data Conversion,” in *IEEE 24th International Conference on Software Analysis, Evolution and Reengineering*, in SANER ’17. 2017, pp. 193–204. doi: [10.1109/SANER.2017.7884621](https://doi.org/10.1109/SANER.2017.7884621).
- [89] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley, 1994.
- [90] D. Leijen, “Algebraic Effect Handlers with Resources and Deep Finalization,” technical report MSR-TR-2018-10, Apr. 2018.
- [91] M. Fowler, “Event Sourcing.” 2005.
- [92] J. Lee, J. Ahn, and K. Yi, “React-tRace: A Semantics for Understanding React Hooks,” *Proc. ACM Program. Lang.*, vol. 9, no. OOPSLA2, pp. 471–498, 2025, doi: [10.1145/3763067](https://doi.org/10.1145/3763067).

- [93] N. Shavit and D. Touitou, “Software Transactional Memory,” in *Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing*, in PODC '95. 1995, pp. 204–213. doi: [10.1145/224964.224987](https://doi.org/10.1145/224964.224987).
- [94] T. Harris, S. Marlow, S. Peyton Jones, and M. Herlihy, “Composable Memory Transactions,” in *Proceedings of the Tenth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, in PPOPP '05. 2005, pp. 48–60. doi: [10.1145/1065944.1065952](https://doi.org/10.1145/1065944.1065952).
- [95] M. Herlihy and J. E. B. Moss, “Transactional Memory: Architectural Support for Lock-Free Data Structures,” in *Proceedings of the 20th Annual International Symposium on Computer Architecture*, in ISCA '93. 1993, pp. 289–300. doi: [10.1145/165123.165164](https://doi.org/10.1145/165123.165164).
- [96] R. Landauer, “Irreversibility and Heat Generation in the Computing Process,” *IBM Journal of Research and Development*, vol. 5, no. 3, pp. 183–191, 1961, doi: [10.1147/rd.53.0183](https://doi.org/10.1147/rd.53.0183).
- [97] C. H. Bennett, “Logical Reversibility of Computation,” *IBM Journal of Research and Development*, vol. 17, no. 6, pp. 525–532, 1973, doi: [10.1147/rd.176.0525](https://doi.org/10.1147/rd.176.0525).
- [98] T. Yokoyama and R. Glück, “A Reversible Programming Language and its Invertible Self-Interpreter,” in *Proceedings of the 2007 ACM SIGPLAN Workshop on Partial Evaluation and Semantics-Based Program Manipulation*, in PEPM '07. 2007, pp. 144–153. doi: [10.1145/1244381.1244404](https://doi.org/10.1145/1244381.1244404).
- [99] P. Wadler, “Linear Types Can Change the World!,” p. , 1993.
- [100] C. Walls, *Spring in Action*, 6th ed. Manning Publications, 2022.
- [101] C. Escoffier, R. S. Hall, and P. Lalanda, “iPOJO: an Extensible Service-Oriented Component Framework,” in *IEEE International Conference on Services Computing*, 2007, pp. 474–481. doi: [10.1109/SCC.2007.74](https://doi.org/10.1109/SCC.2007.74).
- [102] H. Cervantes and R. S. Hall, “Autonomous Adaptation to Dynamic Availability Using a Service-Oriented Component Model,” in *Proceedings of the 26th International Conference on Software Engineering*, in ICSE '04. 2004, pp. 614–623. doi: [10.1109/ICSE.2004.1317483](https://doi.org/10.1109/ICSE.2004.1317483).
- [103] C. Elliott and P. Hudak, “Functional Reactive Animation,” in *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming*, in ICFP '97. 1997, pp. 263–273. doi: [10.1145/258948.258973](https://doi.org/10.1145/258948.258973).
- [104] G. H. Cooper and S. Krishnamurthi, “Embedding Dynamic Dataflow in a Call-by-Value Language,” in *Programming Languages and Systems (ESOP 2006)*, in Lecture Notes in Computer Science, vol. 3924. Springer, 2006, pp. 294–308. doi: [10.1007/11693024\\_20](https://doi.org/10.1007/11693024_20).
- [105] I. Maier and M. Odersky, “Deprecating the Observer Pattern with Scala.React,” technical report EPFL-REPORT-176887, 2012. [Online]. Available: <https://infoscience.epfl.ch/record/176887>
- [106] E. Bainomugisha, A. L. Carreton, T. Van Cutsem, W. De Meuter, and others, “A Survey on Reactive Programming,” *ACM Comput. Surv.*, vol. 45, no. 4, 2013, doi: [10.1145/2501654.2501666](https://doi.org/10.1145/2501654.2501666).

## A 术语表

本术语表只收录本文提出、形式化定义，或在 Cordis 模型中被赋予特殊含义的概念。效应、余效应、依赖注入、热模块替换等已有通用概念，除非本文对其作了专门重构，否则不单独列入。英文词项按正文写法保留；中文译名用于全文统一。

### A.1 可组合性的维度

**时空可组合性** 英文：*spatiotemporal composability*。本文对动态可组合性的总称，由相互正交的时间可组合性与空间可组合性共同构成。这里的“时空”指组件在生命周期时间轴上的可回退性，以及组件在依赖拓扑中的可协调性，并非几何或物理意义上的时空。

**时间可组合性** 英文：*temporal composability*。在运行时移除组件时，完整且安全地回退该组件对共享环境所作修改的能力。本文的形式模型只覆盖系统能够完全控制的内部上下文状态；网络请求、文件 I/O 等外部交互需采用补偿事务等领域机制处理。

**空间可组合性** 英文：*spatial composability*。组件以结构化、可验证的方式声明、发现并解析相互依赖，并在依赖出现、消失或改变身份时协调组件生命周期的能力。

### A.2 可回退效应

**可回退效应** 英文：*revertible effect*。一种运行时效应模型：每次上下文变换都显式给出对应逆函数，运行时在执行期间记录并组合这些逆函数，从而在组件移除时恢复组合前的上下文。*Revertible* 强调“显式提供逆函数并由运行时跟踪”的操作契约；一般的 *reversible* 只表示变换可逆，因此本文分别译为“可回退”与“可逆”。

**效应上下文** 英文：*effect context*。记作  $\partial\Gamma := \Gamma \times \mathfrak{F}_\Gamma$  的运行时上下文，由当前状态  $\gamma$  与累积恢复变换  $\varphi$  组成。 $\varphi$  记录如何把当前状态恢复到该上下文的初始状态。

**效应函数与严格效应函数** 英文：*effect function* 与 *strict effect function*。效应函数  $e : \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)$  在输入状态  $\gamma$  上返回新状态  $\delta$  和候选逆函数  $g$ 。严格效应函数还满足  $g(\delta) = \gamma$ ，即返回的函数确实能撤销本次变换。

**效应组合** 英文：*effect composition*。本文以  $\diamond$  表示效应函数的组合。若先执行  $g$  再执行  $f$ ，则组合结果同时组合二者的逆函数，并以与执行顺序相反的顺序恢复；这一运算使可回退性在复合效应下保持成立。

**效应跟踪与效应恢复** 英文：*effect tracking* 与 *effect recovery*。效应跟踪把每个原子效应返回的逆函数依次合入效应上下文的  $\varphi$ ；效应恢复则把累积的  $\varphi$  应用于当前状态，并把恢复变换重置为恒等函数。本文语境中的“恢复”指重建上下文先前状态，不等同于事务失败后的“回滚”。

**释放函数与幂等防护** 英文：*disposer* 与 *idempotent guard*。释放函数是一次效应执行后交给调用方的局部逆函数，可单独撤销该效应。幂等防护为每个释放函数生成私有句柄，使其至多生效一次；它保护的是返回给调用方的局部逆函数，而非已经具有幂等性的整体恢复操作。

**效应迭代器** 英文：*effect iterator*。多步效应执行的递归表示；每一步返回新上下文、当前步骤的逆函数，以及可选的下一步。步骤边界构成自然的中断点，已完成步骤的逆函数按后进先出顺序组合，因此迁移在中断后仍能安全恢复。

**双向效应迭代器** 英文：*bidirectional effect iterator*。效应迭代器的扩展形式，把单个逆函数替换为另一个迭代器，使正向执行与反向恢复都可分步进行，并可在两个方向间中断切换。论文将其视为理论扩展，因为主流语言很少原生支持双向迭代语义。



### A.3 反应式余效应

**反应式余效应** 英文: *reactive coeffect*。本文把静态余效应提升为运行时依赖机制后的概念。组件声明依赖规格; 共享上下文变化时, 系统重新判断依赖满足性, 并据此自动驱动组件激活、停用与效应恢复。

**余效应上下文** 英文: *coeffect context*。由依赖键到有类型值的有限依赖偏函数, 记作  $\Sigma := (k : K) \mapsto \mathcal{V}_k$ 。它在运行时具体化组件从环境中需要什么, 并通过类型族  $\mathcal{V}$  约束每个键对应的值类型。

**余效应规格** 英文: *coeffect specification*。组件向环境声明的依赖集合, 基本形式为  $\mathcal{D}_\Sigma := \text{Set}(K)$ 。在带拦截的扩展中, 规格还可为各依赖键携带组件声明的元数据。

**依赖满足性与通知** 英文: *dependency satisfaction* 与 *satisfaction-based notification*。当规格  $d$  中的全部键都存在于余效应上下文时, 记作  $\sigma \models d$ 。系统比较变更前后的满足状态, 把迁移分类为激活型、停用型或中性, 并由分类结果触发生命周期动作。

**余效应隔离与隔离域** 英文: *coeffect isolation* 与 *isolation realm*。余效应隔离借助“逻辑键到域标识符”和“域标识符到有类型值”的两层映射, 使同一个依赖键能在不同上下文中解析为不同值。隔离域是该解析机制中的命名空间标识, 不等同于进程、容器或安全沙箱等强隔离边界。

**余效应拦截** 英文: *coeffect interception*。在不替换依赖值的前提下, 把横切元数据附加到依赖访问的机制。组件声明的元数据与上下文携带的元数据在读取时合并, 并交给提供方函数解释; 它改变依赖的使用方式, 而不改变依赖是否存在, 因而通常不会触发组件重载。

### A.4 生命周期与上下文范式

**组件** 英文: *component*。本文形式模型中的运行时实体, 由余效应规格  $d$  与效应函数  $e$  配成一对, 并带有可变的生命周期状态。规格决定组件需要什么, 效应函数决定组件活跃时向上下文贡献什么。

**目标状态** 英文: *target state*。由效应侧的存活条件与余效应侧的满足条件共同决定的期望生命周期状态。组件效应已经应用且声明的依赖全部满足时, 目标状态为 ACTIVE; 否则为 INACTIVE。

**组件生命周期** 英文: *component lifecycle*。把可回退效应与反应式余效应转化为运行时行为的状态模型。RELOAD 执行组件的效应函数并累积逆函数, UNLOAD 应用累积逆函数恢复上下文; 迭代、纪元与惯性语义进一步处理可中断、依赖替换和异步控制流。

**纪元与基于纪元的生命周期** 英文: *epoch* 与 *epoch-based lifecycle*。纪元是针对某一余效应规格取得的已解析依赖值元组, 用于给组件目标状态的具体依赖配置编号; 非活跃状态使用特殊值  $\perp$ 。不同活跃纪元形成从非活跃状态分出的独立分支, 因此“纪元”不是时间区间, 而是依赖配置的版本标识。

**惯性状态** 英文: *inertial state*。异步生命周期中的 RELOAD 与 UNLOAD 迁移状态。一旦进入, 当前迁移先运行至完成, 系统随后才根据届时的目标状态执行或丢弃反向迁移。这里的“惯性”表示迁移不可被外部状态变化立即反转, 不表示惰性求值或静止状态。

**统一上下文类型** 英文: *unified context type*。记作  $\Gamma_\infty := \mu\Gamma. \Gamma \times (\Gamma \rightarrow \Gamma) \times \Sigma$  的递归类型, 同时容纳当前上下文状态、累积逆函数和余效应上下文。它把效应上下文的层级塔压缩为自相似结构, 并让依赖操作与其逆转都经过同一个实体。

**上下文范式** 英文: *context paradigm*。一种通过显式的一等上下文统一中介效应与余效应的编程范式。开发者为原子效应提供逆函数, 并声明组件所需依赖; 运行时负责组合恢复操作、解析依赖和响应式重连, 从而兼顾显式状态传递的可追踪性与命令式调用的易用性。

**就地实现方式与派生实现方式** 英文: *in-place realization* 与 *derived realization*。同一效应指称

的两种宿主语言实现方式。就地方式修改输入上下文并返回非平凡逆函数；派生方式保持输入不变，返回递归结构中的新上下文及恒等逆函数，恢复时直接丢弃派生上下文。

### A.5 Cordis 实现术语

**纤程** 英文：*fiber*。Cordis 中组件的运行时实例，保存组件的静态规格及当前迁移状态，包括父上下文、子上下文、累积释放函数、纪元和正在进行的迁移句柄。它不是并发运行时中与线程相对的轻量执行单元。

**时空可组合性元框架** 英文：*meta-framework of spatiotemporal composability*。论文对 Cordis 的定位：它不规定 Web、聊天机器人或 UI 等具体应用领域，只提供通用的动态组合语义，包括效应跟踪、余效应解析、组件生命周期和声明式加载能力。